Agentic AI Infrastructure

Rico Gross
Faculty of Electronics and Computer Science
Aalen University of Applied Sciences
Aalen, Germany
rico.gross@studmail.htw-aalen.de

***Abstract*—This paper shortly traces the evolution of Artificial Intelligence from its foundational roots in John McCarthy's 1956 vision of symbolic reasoning to the modern emergence of Agentic AI. Then, a strict distinction is made between AI Agents, that are defined as bounded software entities that perceive inputs and produce actions according to predefined objectives within constrained environments, and Agentic AI, which refers to a systemic paradigm characterized by dynamic goal decomposition, multi-step reasoning, tool orchestration, memory persistence and multiple autonomous AI Agents that collaborate to dynamically fulfill complex, multi-faceted goals. This shift toward multi-agent ecosystems introduces the MxN integration problem, a critical scalability bottleneck that occurs in several layers of Agentic AI systems. To resolve this complexity, this paper argues for the adoption of a standardized infrastructure stack. The paper provides an technical analysis of three emerging protocols: Model Context Protocol (MCP), which standardizes how AI Agents access external data, Agent-to-Agent (A2A) protocol, that facilitates negotiation and delegation between individual AI Agents and the Agent-User Interaction (AG-UI) protocol, which provides a standard for how AI Agents communicate with user-facing applications. The paper concludes that these three protocols are not redundant, but strictly complementary, together forming the essential infrastructure protocol stack required to move beyond isolated AI Agents toward a unified, interoperable Agentic AI ecosystem where multiple AI Agents can collaboratively work on complex goals through standardized communication, enabling scalability, adaptability and modularity.**

***Index Terms*—AI Agents, Agentic AI, MxN integration problem, Model Context Protocol (MCP), Agent-to-Agent (A2A) protocol, Agent-User Interaction (AG-UI) protocol**

INTRODUCTION

The field of Artificial Intelligence (AI) was informally established at the 1956 Dartmouth Workshop, where John McCarthy coined the term *Artificial Intelligence*. This period gave rise to a new era of great expectations, fuelled by early successes such as the General Problem Solver [1] and the creation of Lisp [2]. However, the field soon encountered significant challenges, as the methods that were effective in addressing smaller problems proved to be insufficient when applied to more extensive and complex real-world tasks. As of the 1970s, there was a shift in focus towards agent-like systems that were designed to perform specific tasks with predefined rules. These systems offered limited autonomy and minimal adaptability to dynamic environments. The aforementioned systems were primarily reactive or deliberative in nature, with a reliance on symbolic reasoning, rule-based logic, fixed control rules (e.g. Stanford Cart [3]), pattern matching or scripted behaviors through predefined decision trees. In contrast, modern AI Agents exhibit learning-driven, context-aware capabilities [4]. For instance, expert systems employed knowledge bases comprising domain-specific knowledge and inference engines to emulate human decision-making (e.g. MYCIN [5]). As is evident in the extant literature, these early agent-like systems shared common limitations with self-learning, generative reasoning and the adaptability to unstructured or evolving environments [6].

As the new millennium began, there was a shift in focus towards Big Data, leading to a consensus that substantial datasets frequently outweighed complex algorithms. The contemporary epoch is characterized by a profound transformation in the field of machine learning, commonly referred to as the deep learning revolution. The development of automated representation learning has been enabled by the advancement of deep neural networks, which have superseded the limitations of hand-crafted features. This evolution has culminated in the generative AI boom [7], as evidenced by the proliferation of transformer architectures and Large Language Models (LLMs) that have expanded the capabilities of AI from mere classification to human-level creation [6]. The aforementioned shortcomings of early agent-like systems serve to distinguish them from a recently emerging paradigm termed Agentic AI. The latter builds on deep learning, reinforcement learning and foundation models in order to enable AI Agents with contextual awareness, continuous learning and emergent autonomy. Recent interest in Agentic AI from the public, academic and industry sectors is indicative of a more extensive transition in system capabilities [6], [8], [9]. This rise in interest is also reflected in the increased search popularity for this topic on the established search engine Google [10].

Fig. 1 shows the development of search interest in the term "Agentic AI" since February 2024. The popularity values represent search interest in relation to the highest point on the chart. A value of 100 indicates the highest recorded popularity of the term so far. Therefore, a value of 50 means that the term was only half as popular at that point in time.

This evolution is closely tied to the evolution of agent design from the pre-transformer era, where early agent-like systems operated in constrained, rule-based environments, to the post-LLM period marked by learning-driven, adaptive architectures. These contemporary systems empower AI Agents to enhance their performance over time and interact autonomously with unstructured, dynamic inputs. For instance, while early agent-like systems necessitated manual updates to static knowledge

bases, contemporary AI Agents employ emergent neural architectures to generalize across tasks. The increasing prevalence of these Google searches can be attributed to the growing awareness of this technological advancement, as researchers and practitioners seek tools that extend beyond automation, aiming to achieve autonomy and general-purpose reasoning. Furthermore, applications have expanded beyond specialized domains such as simulations or logistics, encompassing a broad spectrum of practical settings that necessitate real-time reasoning and adaptive control. This momentum, as visualized in Fig. 1, highlights the significance of recent architectural advances in scaling autonomous AI Agents for real-world deployment. The release of ChatGPT [11] in 2022 signified a pivotal inflection point in the development and public perception of AI, catalyzing a global surge in adoption, investment and research activity. Consequently, the AI landscape has undergone a rapid transformation, shifting from the use of generative AI toward more autonomous, task-oriented AI systems. This evolution proceeded through two major post-generative phases: AI Agents and Agentic AI.

The initial widespread success of ChatGPT [11] led to the popularization of generative agents, which are LLM-based systems designed to produce novel outputs such as text, images and code from user prompts. These were rapidly adopted across a range of applications, including conversational assistants (e.g. GitHub Copilot [12]), content-generation platforms (e.g. Jasper [13]) and creative tools (e.g. Midjourney [14]), thereby bringing about a paradigm shift in domains such as digital design, marketing and software prototyping. This generative foundation subsequently gave rise to a novel class of systems, commonly designated as AI Agents. These AI Agents augment LLMs with capabilities for external tool use, function calling and sequential reasoning, thereby empowering them to retrieve real-time information and execute multi-step workflows autonomously. This transition is exemplified by frameworks such as AutoGPT [15] and BabyAGI [16]. By late 2023, significant advancements had been made in the field of Agentic AI. This emerging field encompasses the use of sophisticated, multi-agent systems, wherein specialized AI Agents collaborate to decompose objectives, facilitate communication and orchestrate actions in pursuit of overarching goals [8], [17]–[22].

In line with this evolution, three standardized protocols emerged that together build a foundational infrastructure these modern Agentic AI systems build upon. The protocols Model Context Protocol (MCP) [23], Agent-to-Agent (A2A) protocol [24] and Agent-User Interaction (AG-UI) protocol [25] are designed to enable seamless interoperability across the whole Agentic AI system to shift the focus fully to the development of the single components of the system and away from the necessary infrastructure logic to compose the whole system.

A. Overview

This paper provides a comprehensive analysis of the emerging communication standards required to transition from isolated AI entities to scalable, collaborative agentic ecosystems.

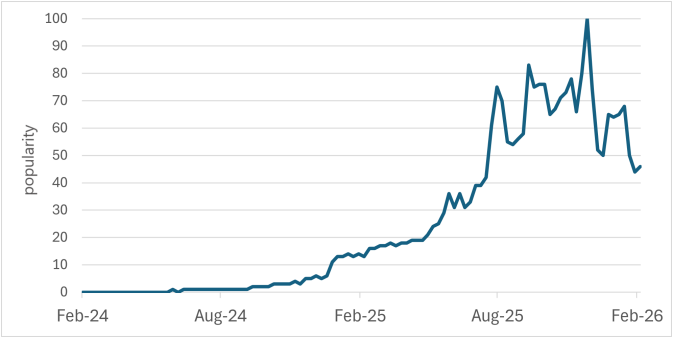


Fig. 1. Global Google search trend showing rising interest for the term "Agentic AI", especially since February 2025 [10].

While section I clarifies the distinction between AI Agents, defined as autonomous, tool-augmented software entities and Agentic AI, characterized by orchestrated multi-agent collaboration and emergent system-level intelligence, the primary focus of this paper lies on the technical infrastructure enabling this integration. Despite the rapid advancement of LLMs, a significant "integration crisis" has emerged, described herein as the MxN integration problem in section II. It occurs if an increasing number of specialized AI Agents must interact with an expanding universe of data sources, interfaces and other AI Agents, resulting in a fragmented landscape of bespoke glue code that is inherently non-scalable and difficult to maintain. To address these bottlenecks, this paper evaluates three pivotal protocols that collectively form a standardized infrastructural backbone for the next generation of intelligent Agentic AI systems. First, section III examines the Model Context Protocol (MCP), a model-agnostic standard designed to decouple AI Agents from their underlying data sources, effectively replacing fragmented connectors with a universal interface for tool invocation and context sampling. Building upon this, section IV dives into the Agent-to-Agent (A2A) protocol, which facilitates secure, structured interoperability between independent AI Agents built on disparate frameworks. By treating AI Agents as opaque, autonomous entities, A2A enables cross-vendor collaboration while preserving intellectual property and reducing the complexity of multi-agent orchestration. Finally, section V explores the Agent-User Interaction (AG-UI) protocol, a event-based communication standard that moves beyond the limitations of traditional request-response REST APIs by enabling real-time, bidirectional streaming of agentic actions to frontend applications, ensuring a transparent and collaborative user experience. By synthesizing these technical standards, the paper offers a structured taxonomy and a forward-looking roadmap for the deployment of Agentic AI systems in domains such as robotics, healthcare and enterprise automation. This paper aims to provide researchers and developers with actionable insights into how this infastructural protocol stack effectively mitigates integration complexity and unlocks the full potential of orchestrated agentic intelligence in Agentic AI systems.

I. The emergence of Agentic AI

A recurring theme in the extant literature is the conceptualization of generative AI as the foundational precursor to agentic intelligence. The operation of these systems is primarily reliant upon pre-trained LLMs and Large Vision Models (LVMs, e.g. CLIP [26]). These effectively facilitate the synthesis of multi-modal content, encompassing text, images, audio and code, in response to input prompts. Although highly communicative, generative models demonstrate a high degree of reactivity. That is to say, they generate output exclusively in response to explicit prompts and do not engage in autonomous pursuit of goals or self-initiated reasoning [27], [28].

Despite their noteworthy capacity for generative fidelity, these systems are constrained by their inability to act upon the environment or manipulate digital tools independently. For instance, they are unable to search the internet, parse real-time data or interact with APIs without the use of human-engineered wrappers. These limitations of generative AI in handling static knowledge cutoffs, restricted interaction scopes, dynamic tasks, maintaining state continuity or executing multi-step plans led to the development of tool-augmented systems, commonly referred to as AI Agents. These systems augment the language processing foundation of LLMs with additional infrastructure components, including memory buffers, tool-calling APIs, reasoning chains and planning routines. This augmentation aims to bridge the gap between passive response generation and active task completion. This architectural evolution, marked by the integration of perception, decision-making and external tool-use within closed feedback loops, signifies a pivotal shift in the design of AI systems: from content creation to autonomous task execution. This enables AI Agents to access real-time information, execute code and interact with dynamic environments. By now, a prominent example of such a tool-augmented AI Agent is ChatGPT [11], which meanwhile autonomously invokes a web search API when unable to answer a query directly to retrieve more recent and relevant information, performs reasoning over the retrieved content and formulates a response [8], [22], [28], [29]. This transition from generative systems to AI Agents exemplifies a progressive layering of functionality that, in essence, supports the emergence of agentic behaviors [8].

As the field progresses from generative agents toward increasingly autonomous systems of Agentic AI, it becomes critically important to delineate the technological and conceptual boundaries between AI Agents and Agentic AI. While both paradigms build upon LLMs and extend the capabilities of generative systems, they embody fundamentally different architectures, interaction models and levels of autonomy [8], [9].

A. AI Agents

AI Agents can be defined as autonomous software systems engineered for goal-directed task execution within bounded environments. These AI Agents are distinguished by their capacity to perceive structured or unstructured inputs, to reason over contextual information and to initiate actions toward the achievement of specific objectives. They frequently function as surrogates for human users or subsystems. Conventional automation scripts, by contrast, adhere to deterministic workflows. On the contrary, AI Agents exhibit reactive intelligence and a degree of adaptability, enabling them to interpret dynamic inputs and adjust their outputs accordingly. The adoption of this technology has been documented in a variety of application domains, including customer service automation, personal productivity assistance, organizational information retrieval and decision support systems [8], [22], [30], [31].

AI Agents are frequently conceptualized as instantiated operational instances of artificial intelligence. These AI Agents are designed to interface with users, software ecosystems or digital infrastructures to develop goal-directed behavior. AI Agents are distinguished from general-purpose LLMs in three key ways. First, they exhibit structured initialization, meaning they are characterized by a defined sequence of actions or procedures that occur at the onset. Second, they demonstrate bounded autonomy, indicating that they are able to make decisions and act within specific limits or constraints. Third, and finally, they exhibit persistent task orientation, which refers to their ability to maintain focus on the objectives of a given task over an extended period. LLMs predominantly function as reactive prompt followers, whereas AI Agents operate automatically within explicitly defined scopes, engaging dynamically with inputs and producing actionable outputs in real-time environments. A review of the extant literature reveals three foundational characteristics that are commonly incorporated by architectural taxonomies and empirical deployments of AI Agents. It has been demonstrated that these traits, when considered collectively, establish the foundational principles for agent design. These AI Agents are capable of exhibiting goal-oriented, adaptive behaviors within the confines of specific environments. This capacity distinguishes them from general-purpose or generative models [8], [9], [22], [32].

1) Autonomy: A fundamental aspect of AI Agents is their capacity to operate with minimal or no human intervention following deployment. Subsequent to initialization, these AI Agents possess the capacity to perceive environmental inputs, engage in reasoning over contextual data and execute predefined or adaptive actions in real time. Autonomy facilitates scalable deployment in applications where persistent oversight is impractical, such as customer support bots or scheduling assistants.

2) Task-Specificity: AI Agents are engineered to execute specific and precisely delineated tasks. These systems have been engineered to carry out repeatable operations within a constrained domain. Such operations can include email filtering, database querying or calendar coordination. This task specialization facilitates efficiency and high precision in automating tasks where general-purpose reasoning is unnecessary or inefficient.

3) Reactivity and Adaptation: AI Agents frequently incorporate fundamental mechanisms for interacting with dynamic

inputs, enabling them to respond to real-time stimuli such as user requests, external API calls or state changes in software environments. In certain systems, fundamental learning capacities are integrated through feedback loops, heuristics or updated context buffers. These mechanisms serve to refine behavior over time, particularly in contexts such as personalized recommendations or conversation flow management.

These core characteristics collectively enable AI Agents to serve as modular, lightweight interfaces between pretrained LLMs or LVMs and domain-specific utility pipelines. Their architectural simplicity and operational efficiency position them as key enablers of scalable automation across enterprise, consumer and industrial settings [9], [22], [33]–[35].

The remained of this section dives into the importance of foundational models regarding AI Agents. The advent of LLMs and LVMs, which function as the core reasoning and perception engines in contemporary agent systems, has led to a substantial acceleration in the progress of AI Agents. These models empower AI Agents to interact intelligently with their environments, comprehend multi-modal inputs and execute complex reasoning tasks that surpass the limitations of hard-coded automation.

Contemporary LLMs are trained on extensive text corpora, encompassing books and web content, through the utilization of self-supervised objectives. These models undergo fine-tuning through techniques such as Supervised Fine-Tuning (SFT) and Reinforcement Learning from Human Feedback (RLHF) [36]. These models encode rich statistical and semantic knowledge, allowing them to exhibit emergent capabilities such as natural language understanding, question answering, inference, summarization, code generation and dialogue management. However, in agentic contexts, their capabilities extend beyond response generation. In the context of AI Agents, LLMs function as cognitive engines that interpret user objectives, formulate and evaluate potential action plans, plan multi-step solutions, select the most suitable strategies, utilize external tools, generate human-like responses and manage complex, multi-step workflows. Recent studies have identified these models as being pivotal to the architecture of AI Agents. These models are responsible for analyzing high-level objectives, decomposing them into actionable subtasks, invoking external APIs as required and monitoring progress to determine subsequent actions. In such systems, the LLM functions in a recurrent cycle comprising prompt processing, state update and feedback-based correction, thereby closely emulating autonomous decision-making processes [8], [17], [22].

Large Vision Models (LVMs), including CLIP [26], have been demonstrated to extend the capabilities of AI Agents into the visual domain. These are trained on image-text pairs, thereby enabling perception-based tasks such as image classification, object detection and vision-language grounding. These capabilities are becoming increasingly vital for AI Agents operating in domains such as robotics, autonomous vehicles and visual content moderation [18], [32].

For example, as illustrated in Fig. 2, when an autonomous drone AI Agent is tasked with monitoring orchards, a LVM can identify diseased fruits [37] or damaged branches by interpreting live imagery. Upon detection, the system autonomously triggers predefined intervention protocols, such as notifying staff or marking the location for targeted treatment without requiring human involvement. This workflow exemplifies the autonomy and reactivity of AI Agents in agricultural environments [22].

Importantly, LLMs and LVMs are often accessed via APIs provided by cloud-based platforms such as OpenAI [38], Anthropic [39] and Google Gemini [40]. These services abstract away the complexity of model training and fine-tuning, enabling developers to rapidly build and deploy AI Agents equipped with state-of-the-art reasoning and perceptual abilities. This integrability accelerates prototyping and allows agent frameworks like LangChain [41], LangGraph [42] and AutoGen [43] to orchestrate LLM and LVM outputs across task workflows. In short, foundational AI models give modern AI Agents their basic understanding and allow them to make smart decisions in complex situations.

The emergence of these capabilities has paved the way for more sophisticated behavior patterns in AI Agents, including long-horizon planning, cross-tool coordination and adaptive learning loops. However, the integration of these tools introduces novel challenges, including coordination complexity, error propagation and context window limitations. The progression toward AI Agents is inseparable from the strategic integration of LLMs as reasoning engines and their augmentation through structured utilization of external tools, such as search engines and APIs. This synergy transforms static language models into dynamic cognitive AI Agents capable of perceiving, planning, acting and adapting, thus establishing the foundation for multi-agent collaboration, persistent memory and scalable autonomy, which are the characteristics of Agentic AI systems [8], [9], [22], [32], [35].

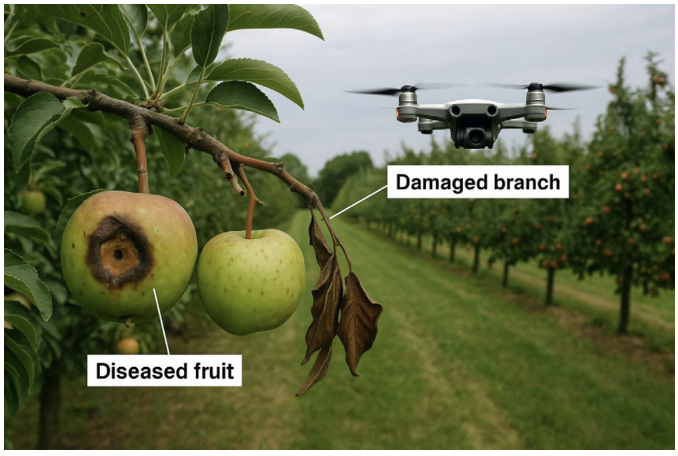


Fig. 2. Drone AI Agent autonomously inspecting an orchard, identifying diseased fruits and damaged branches using LVMs, triggering respective alerts [22].

B. Agentic AI

While AI Agents represent a substantial advancement in AI capabilities, particularly in automating specific tasks through tool-augmented reasoning, recent literature identifies notable limitations that constrain their scalability in complex, dynamic, multi-step and/or cooperative scenarios. As explored in the prior section, AI Agents employ LLMs in conjunction with external tools and APIs to execute operations with a narrow scope, including responding to customer inquiries, retrieving documents and managing schedules. However, as use cases increasingly demand context retention, task interdependence and adaptability across dynamic environments, the single-agent architecture, in which a single AI Agent attempts to fulfill a specific use case, proves insufficient [8], [9], [17], [22], [44].

These constraints have catalyzed the development of a more advanced paradigm: Agentic AI. Agentic AI systems represent an emergent class of intelligent architectures that extend the capabilities of AI Agents by enabling multiple of these to collaboratively pursue goals through structured communication, shared memory and dynamic role assignment. In Agentic AI systems multiple specialized AI Agents collaborate to achieve complex, high-level objectives utilizing collaborative reasoning and multi-step planning. Within the context of such systems, each modular AI Agent is assigned a specific subcomponent of a broader objective, with coordination facilitated by either a centralized orchestrator or a decentralized protocol. This structural change indicates a conceptual departure from the individual, reactive behaviors typically observed in single-agent architectures, toward a form of system-level intelligence characterized by dynamic inter-agent collaboration. The goal decomposition paradigm is a key enabler in this context, since it involves the automatic parsing and division of a user-specified objective into smaller, manageable tasks. Subsequently, these subtasks are then allocated across the agent network. Multi-step reasoning and planning mechanisms facilitate the dynamic sequencing of these subtasks, allowing the system to adapt in real time to environmental changes or partial task failures. This agentic architecture ensures robust task execution even under uncertainty. Furthermore, reflective reasoning and memory systems enables storing of context across multiple interactions, evaluating past decisions and iteratively refining strategies. The aforementioned capabilities, when considered collectively, enable Agentic AI systems to exhibit flexible, adaptive, cooperative and collaborative intelligence. This intelligence exceeds the operational limits of individual AI Agents.

In the domain of robotics and automation, Agentic AI facilitates collaborative behavior in multi-robot systems. For instance, each robot functions as a task-specialized AI Agent, such as a picker, a transporter or a mapper. Concurrently, an orchestrator oversees and adapts workflows in a commercial apple orchard, as illustrated in Fig. 3. In this particular case, Agentic AI enables the coordination of a multi-robot system, with the objective of optimizing fruit harvesting. The initial survey of the orchard is conducted by mapping drones, resulting in the generation of high-resolution yield maps and the identification of ripe fruit clusters. This spatial data is disseminated through a centralized memory layer, accessible by all AI Agents. Subsequently, picker robots are assigned to high-density zones, guided by path-planning AI Agents that optimize routes around obstacles and labor zones. Transport AI Agents dynamically transport fruit containers between pickers and storage, adjusting tasks in response to picker load levels and terrain changes. The coordinator is responsible for the continuous adjustment of task priorities, a process informed by weather forecasts and mechanical faults. In the event that a single picker encounters difficulties, nearby units are capable of autonomously reallocating their respective workloads. The coordination exhibited by this system is characterized by its adaptability and memory-driven functionality and exemplifies the potential of Agentic AI to reduce labor costs, increase harvest efficiency and respond to uncertainties in complex agricultural environments, thus surpassing conventional agricultural robots [8], [9], [17], [22], [34], [35].

One of the major challenges of the emerging paradigm of Agentic AI lies in achieving efficient communication across multiple autonomous AI Agents. They must collaboratively pursue a shared objective requiring precise alignment, synchronized execution and robust communication protocols. Also, each AI Agent must be able to access external resources in some way. Without a universal dialect for direct peer-to-peer exchange, AI Agents often experience "lost in translation" errors that derail complex multi-step workflows. Furthermore, the massive overhead of building bespoke connectors for every external system necessitates a unified bridge for context sharing, while a standardized interaction layer is vital for AI Agents to fluently navigate and manipulate the dynamic digital environments they are tasked to manage. Collectively, these bottlenecks highlight the pressing need for standardized communication protocols to provide the infrastructure of modern Agentic AI systems [8], [18], [22], [32].

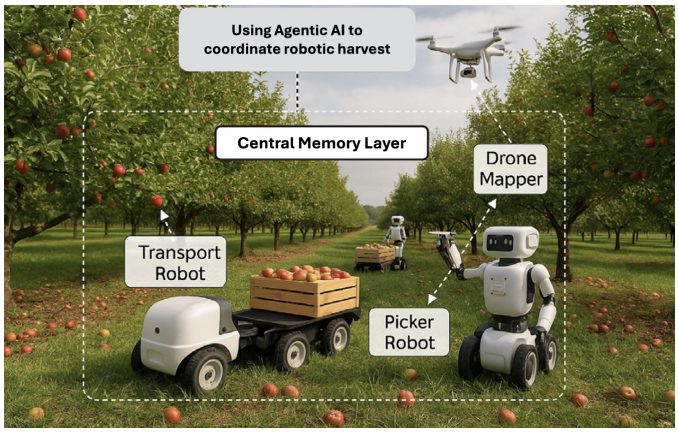


Fig. 3. Agentic AI system coordinating multi-robot harvesting in apple orchards using shared spatial memory and task-specific AI Agents for mapping, picking and transport [22].

II. The MxN integration problem

The MxN integration problem is a significant challenge in the infrastructure for AI ecosystems. Generally, it occurs when multiple clients are connected to several servers. In the absence of a standardized method for establishing connections, each client is required to implement a bespoke integration that functions as an adapter to each server. This suggests the necessity of constructing M multiplied by N integrations. Consequently, a substantial investment of effort is necessary to address these ramifications, which impedes innovation, is costly, repetitive, challenging to maintain and becomes increasingly difficult to scale as the respective ecosystem expands [45]–[48].

Fig. 4 visualizes this problem for a scenario involving four clients that are connected to three servers. This problem has become critically acute with the rise of Agentic AI. As the contemporary industry transitioned from rudimentary chatbots to sophisticated Agentic AI systems, where autonomous AI Agents engage in not only dialogue but also actively perceive, reason and execute tasks, the MxN problem became untenable. The issue no longer pertains to the connection of a single model to a specific tool, rather, it concerns the integration of ecosystems of specialized AI Agents with potentially hundreds of disparate data sources, user interfaces or other AI Agent ecosystems. Thus, in the realm of Agentic AI, the integration problem can manifest across three distinct layers of interaction. Three specific standardized protocols emerged to disassociate integrations within these layers. Instead of building connections between every client and server, these protocols allow both entities to communicate through the same foundation. By this, the problem can effectively be reduced from MxN connections to just M+N, introducing linear complexity in all of these layers. Then, each client connects to the server using the respective protocol, while each server complies to the protocol as well. The subsequent listing presents the three layers of interaction and the protocol that can be utilized in the specific layer to solve the MxN integration problem:

1) Agent $\leftrightarrow$ Tool: This layer utilizes the Model Context Protocol (MCP) [23] to address the data layer. It acts as a universal standard that connects AI Agents to external data sources and tools. Instead of hard-coding API calls for specific services, data sources run a MCP Server that exposes resources and prompts, allowing any MCP-compliant AI Agent, acting as a client, to instantly interface with tools like Google Drive, Slack or local databases without bespoke integration code.

2) Agent $\leftrightarrow$ Agent: This layer relies on the Agent-to-Agent (A2A) protocol [24] to standardize how AI Agents discover, handshake and delegate tasks to one another. It transforms isolated AI Agents into a collaborative mesh, enabling them to negotiate work via the standardized protocol without needing access to the peer's internal logic or having to write agent-specific integration code.

3) Agent $\leftrightarrow$ UI: This layer is managed by the Agent-User Interaction (AG-UI) protocol [25] to solve the integration problem for the communication between possibly several frontends to multiple AI Agents. It provides a standardized way for AI Agents to communicate with frontend applications by instead of simply streaming raw text tokens, AG-UI streams structured events (such as state deltas or tool calls). This decouples the "backend" agent logic from the frontend logic.

The availability of these protocols has the potential to transform the manner in which we conceptualize AI systems. This approach is indicative of scalability, as the addition of a new server does not necessitate new integrations for each client. It means interoperability, because different clients from different ecosystems can share access to the same capabilities. Also, this approach eliminates the need for developers to recreate integrations each time, thereby reducing the overall maintenance burden. Moreover, it opens up space for innovation, since teams are able to prioritize the development of new features rather than the mere act of wiring components together. Fig. 5 visualizes how introducing a standardized protocol to a situation where the MxN integration problem would occur, prevents the combinatorial explosion of necessary integrations by only requiring the clients and servers to apply to the respective protocol. With that, the number of integrations drops to M+N, in this specific case, to seven [45]–[48]. By solving the MxN integration problem for different layers of interaction, these protocols form the new protocol stack for the infrastructure of Agentic AI systems, providing developers with protocols on which to rely on, turning complexity into simplicity and enabling a new era of connected, scalable and collaborative Agentic AI systems [45]–[48].

In the following sections, all three of these protocols will be focused on with more detail.

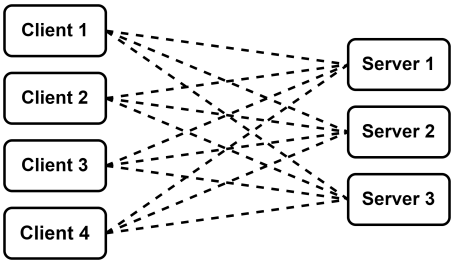


Fig. 4. MxN integration problem. With having 4 clients and 3 servers these are connected to, 12 separate integrations are necessary.

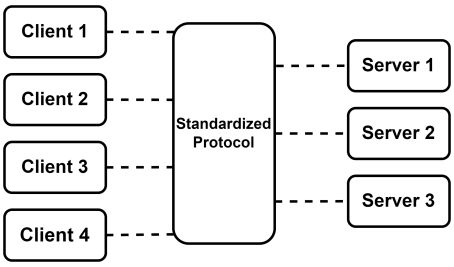


Fig. 5. Standardized protocols solve the MxN integration problem by reducing the number of integrations. With having 4 clients and 3 servers these are connected to, 7 standardized integrations of the protocol are necessary.

III. Model Context Protocol

As AI systems have gained widespread adoption, the industry has initially invested significantly in enhancing model capabilities and prompt techniques, thereby achieving rapid advancements in reasoning and quality. Nonetheless, even the most advanced AI systems are limited by the isolation of the underlying models. This limitation stems from the fact that models are severely constrained by their separation from real-world tools and data sources. With LLMs serving as reasoning engines, the systems surrounding them must facilitate the real-time aggregation and transformation of data from diverse and often heterogeneous sources. This encompasses secure interactions with file systems, web services, databases, development pipelines and even physical sensors. The integration of diverse prebuilt and custom tool sets frequently introduces systemic vulnerabilities, leading to recurrent errors in both tool execution and the subsequent parsing of outputs. Given the necessity of custom implementations for each new data source, the development of truly connected systems, as exemplified by Agentic AI, poses significant challenges in terms of scalability. These architectural complexities and reliability issues represent a substantial barrier to entry, often impeding the practical implementation and deployment of AI systems. In order to orchestrate such interactions, a protocol must not only support declarative capability exchange and structured context injection, but it must also do so while maintaining strict security boundaries, minimal latency and maximal interoperability. These challenges underscore the necessity for standardized tool integration, thereby enabling AI systems to operate autonomously and adapt dynamically to real-world environments [49]–[53].

MCP addresses these challenges by providing a universal, open and model-agnostic standard for connecting AI systems with existing data sources. This replaces fragmented integrations with a single protocol, thus creating a more sustainable architecture. Consequently, this enables any AI model to utilize MCP while any developer or company is permitted to create MCP integrations without requiring explicit authorization. Rather than maintaining separate connectors for each data source, developers can construct their systems around a standard protocol to establish secure, two-way connections between their data sources and AI systems. The result is a simpler, more reliable way for AI systems to maintain context as they access different tools and data sources through a standardized protocol, effectively eliminating the fragmented landscape of tool integrations. Presently, there are thousands of MCP servers available. Popular ones include connectors for Google services (Drive, Gmail, Calendar), Slack (chat and file access), GitHub/Git (for code repositories) and databases like Postgres. The network effect inherent to MCP further enhances its appeal. The utility of MCP is maximized when its users adopt the standard, as the more tools available via MCP, the more advantageous it becomes. MCP is rapidly maturing into a robust standard protocol, thereby transforming AI from an isolated "brain" into a versatile operator. By streamlining the connection of AI Agents with external systems, the way is paved for more capable, interactive and user-friendly AI systems. By that, MCP lays out a standardized way for how AI systems like ChatGPT [11] can find, connect to and use external systems, whether for querying a database, reading from a local file or running a web search through a search engine. This lets models go beyond their training data, making them more flexible and aware of the world in which they operate. With these capabilities, MCP complements agent orchestration frameworks like LangChain [41], LangGraph [42] and crewAI [54]. Generally, MCP introduces a robust and extensible mechanism for structured, stateful communication between clients, hosts and servers. Unlike traditional request-response mechanisms, MCP facilitates bidirectional, session-aware exchanges with support for dynamic capability negotiation, contextual sampling, tool invocation and prompt completion. These interactions build on the well-defined JSON-RPC 2.0 protocol, offering deterministic behavior, fine-grained authorization and adaptive session lifecycle management. Its layered architecture fosters modular design and encourages extensibility. This makes MCP a compelling foundation for building next-generation AI Agents and Agentic AI systems with confidence and control [23], [49]–[53].

A. Architecture

MCP employs a client-server architecture, wherein a MCP host (e.g. an AI Agent) establishes connections with one or more MCP servers (e.g. a server that provides access to the local file system). The MCP host accomplishes this by creating one MCP client for each MCP server it wants to connect to. Each MCP client maintains a dedicated connection with its corresponding MCP server.

Fig. 6 illustrates the high-level overview of this architecture. The figure also includes labeled connections with the respective transport mechanism used for that connection, see section III-C. As an example for this architecture, Visual Studio Code can act as an MCP host. Upon establishing a connection to a MCP server, such as the Sentry MCP server, Visual Studio Code's runtime initiates the creation of an MCP client object, thereby maintaining the connection to the server. Subsequent connections of Visual Studio Code to an alternate MCP server, such as a local file system server, result in the instantiation of an additional MCP client object by the Visual Studio Code runtime to maintain the connection [23], [50], [52].

Below, the finer mechanics of each component of the architecture is be explained to illustrate how they orchestrate the seamless flow of data and actions.

1) Host: The host component within the MCP architecture functions as the pivotal orchestrator and container, responsible for overseeing the interactions between clients and servers and managing their overall life cycle. It operates as the central entity that coordinates secure communication, session management and integration of context across various servers and clients. By effectively abstracting the complexity of server-client interactions, the host simplifies the task of the application developer, allowing developers to focus on their core

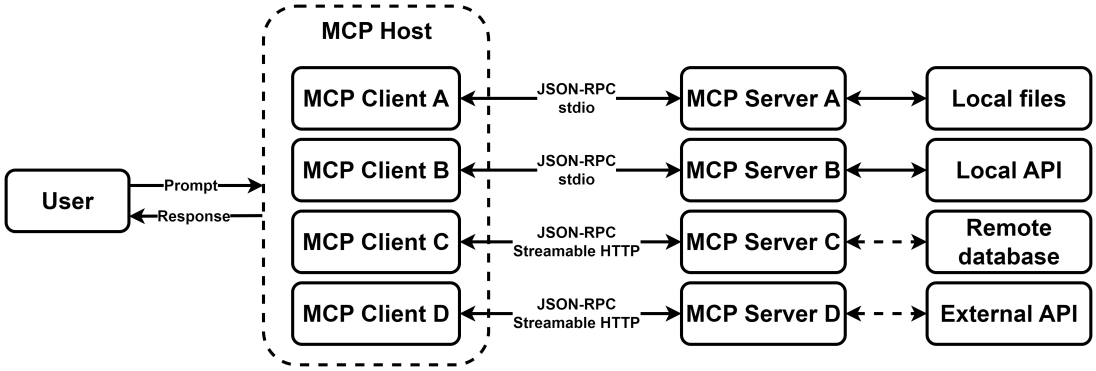


Fig. 6. Overview of the high-level architecture of MCP.

business logic rather than getting entangled in infrastructural complexities. The primary responsibility of the host is the creation and management of multiple client instances, each of which is customized to interact with particular servers. This approach ensures isolation, with clients serving as individual entities maintaining unique stateful sessions. Another fundamental responsibility of the host is the management of the lifecycle of each client, from initialization to operation, through to shutdown. During the initialization phase, the host coordinates capability negotiation, ensuring that both clients and servers are aware of the features they support. This process streamlines subsequent interactions. During normal operation, the host continuously monitors sessions for health and responsiveness, employing utilities such as ping and progress notifications to ensure optimal performance and responsiveness. In the event of anomalies or inactivity, the host is equipped with the capability to terminate sessions, thereby freeing up resources and maintaining the integrity of the overall system. By implementing rigorous authentication protocols and secure token management, the host plays a pivotal role in maintaining the security posture of the entire MCP system. The centralization of orchestration and security enforcement enables the host to significantly reduce the complexity faced by developers in managing multiple client-server interactions. Its role as a container and coordinator ensures robust, secure and efficient management of context and interactions within MCP, ultimately driving greater productivity and innovation in AI-driven application development [23], [50], [52], [53], [55].

2) Server: Servers can be conceptualized as repositories of ready-made tools that an AI Agent can utilize, thereby accessing external contexts. Such a server exposes its specific capabilities through the MCP protocol to facilitate the development of sophisticated, context-aware AI interactions. Rather than constructing monolithic integrations, it is possible to deploy specialized servers that each handle a particular external system. Examples of MCP server integrations include Slack, GitHub, Git, Docker and web search. These servers are typically GitHub repositories available in various programming languages and provide access to specific tools, resources or prompts. These servers have the capacity to execute either locally or remotely. The architecture of MCP servers places significant emphasis on modularity, composability and extensibility, ensuring streamlined, secure and effective interactions. Each server performs a discrete function, allowing developers and applications to leverage specialized functionalities seamlessly within an interoperable ecosystem. Furthermore, this specialization results in maintainable and reusable components. Capability negotiation within MCP ensures compatibility and interoperability, allowing servers to advertise supported features explicitly among diverse client implementations. Servers adhere strictly to defined protocol standards, guaranteeing seamless communication with MCP clients. Moreover, security is a foundational aspect of MCP servers, rigorously enforced through robust authorization and authentication mechanisms. Consequently, the implementation of controlled access to sensitive resources and capabilities is a critical measure to ensure effective mitigation of unauthorized access risks and potential security breaches. Another fundamental design principle is server isolation, as it establishes strict boundaries that prevent access to other contexts offered by different servers. This further supports secure multi-tenant environments and reinforces overall system integrity by allowing each server to function independently within its defined scope. Servers and their corresponding capabilities can be automatically detected by AI systems, without the need for hard-coded integrations. After running a server, AI systems are capable of recognizing and utilizing it through standardized methodologies [23], [50], [52], [53], [55].

As mentioned, servers themselves expose their capabilities. This is facilitated through three primitives:

1) Tools: Tools provide LLMs with direct, model-driven capabilities to interact and execute actions within external systems or data environments. These are defined as LLM-callable functions on the server side, each equipped with typed JSON schema-based input parameters and explicit documentation. These tools expose traditional function calls, but they are standardized through the MCP protocol in such a way, so that the LLM can invoke them based on context. A tool could fetch data via an external API call or update a database, for instance. It is important to emphasize that

MCP does not dictate the timing or the rationale behind a tool's utilization. MCP's primary function is to facilitate a standardized connection, thereby promoting the streamlined integration of tools. In essence, the LLM determines the appropriate tools to invoke based on the context provided by the user's request. By empowering LLMs to autonomously invoke these functionalities, tools substantially augment the practical application and decision-making capabilities of connected AI systems. Despite their autonomous nature, tool invocation might include security checks, enforced through human-in-the-loop mechanisms where the user must accept a certain tool-use. Servers proactively promote available tools through standardized methods, such as *tools/list* and *tools/call*, thereby providing comprehensive discovery and invocation capabilities for LLMs. This transparency enables LLMs to leverage these capabilities in a context-dependent manner, thereby enhancing interaction quality and task automation in complex operational scenarios. Fig. 7 visualizes the message workflow throughout the MCP architecture in relation to certain tools exposed by a server. Available protocol operations for tools include:

- *tools/list* - used by the client to discover all available tools of a server. Return value is an array of tool definitions and the respective JSON schema.
- *tools/call* - used to execute a specific tool.

In a exemplified travel-planning scenario, the AI system might use several tools to book a vacation. A certain MCP server for that scenario might expose a *searchFlights* tool, as in Listing 1.

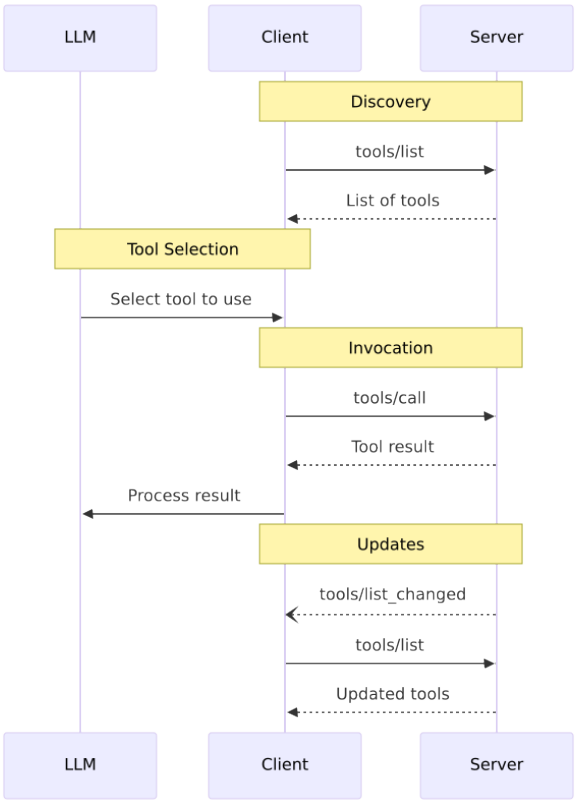


Fig. 7. Message workflow for tools exposed by a server throughout the MCP architecture [23].

```
{
  "name": "searchFlights",
  "description": "Search for available flights",
  "inputSchema": {
    "type": "object",
    "properties": {
      "origin": { "type": "string", "description": "Departure
          city" },
      "destination": { "type": "string", "description": "
          Arrival city" },
      "date": { "type": "string", "format": "date", "
          description": "Travel date" }
    },
    "required": ["origin", "destination", "date"]
  }
}
```

Listing 1. Tool definition for flight searching [23].

2) Resources: Resources are defined as fundamental server components that facilitate the delivery of structured and unstructured data, which is crucial for enhancing LLM interactions. These resources are uniquely identified by URIs (e.g. *file://path/to/document.md*) and can include a variety of content types such as plain text, binary files, images, audio, video and structured documents. The fundamental insight is that while the resource interface is designated as read-only, the underlying data can in fact be dynamic. Examples include files, database records or API responses. The utilization of resources as opposed to tools results in a separation of concerns that is of a higher quality, in a manner analogous to the way in which REST APIs differentiate between data retrieval and data modification endpoints. Resources are application-controlled primitives, allowing diverse strategies for context incorporation, from explicit user-driven resource selection to automatic contextual enrichment based on internal application heuristics. Servers facilitate resource discovery and retrieval via standardized methods such as *resources/list* and *resources/read*, ensuring efficient access and handling. Server implementations can also support subscription mechanisms (*resources/subscribe*), thereby facilitating real-time notifications of resource changes. Fig. 8 shows the message workflow throughout the MCP architecture in relation to certain resources exposed by a server. Available protocol operations for tools include:

- *resources/list* - used by the client to discover all available resources of a server. Return value is an array of resource descriptors.
- *resources/templates/list* - used to discover resource templates.
- *resources/read* - used to retrieve contents of a specific resource.
- *resources/subscribe* - used to monitor resource changes.

Within the exemplified travel-planning scenario, the AI system might use several available resources to retrieve needed travel documents, such as passports. A certain MCP server for that scenario might expose a *travel-documents* resource, as in Listing 2.

3) Prompts: Prompts function as templates and workflows,

```
{
    "uriTemplate": "file://Documents/Travel/{filename}",
    "name": "travel-documents",
    "title": "Travel Documents",
    "description": "Accesses important travel documents such
        as passports, visa or tickets.",
    "mimeType": "application/pdf"
}
```

Listing 2. Resource definition for travel documents [23].

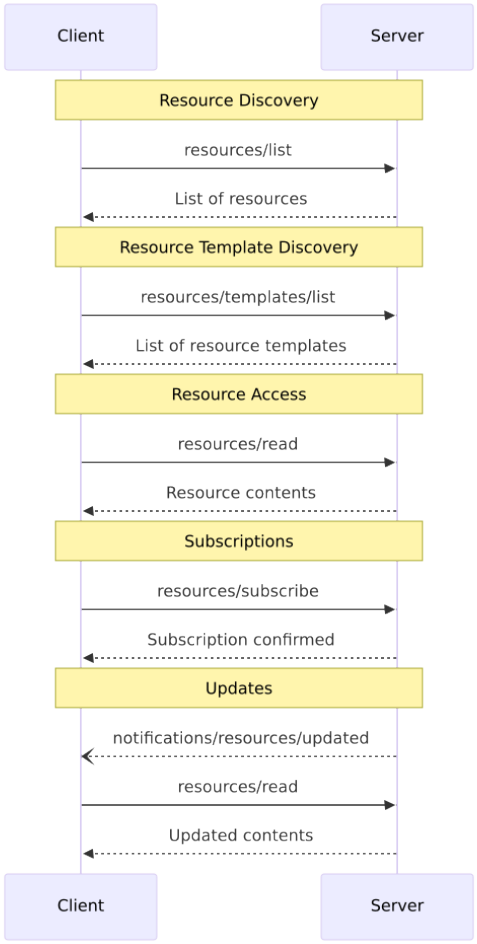


Fig. 8. Message workflow for resources exposed by a server throughout the MCP architecture [23].

guiding interactions with LLMs, significantly enhancing consistency, efficiency and clarity in user-driven engagements. These prompts shift the burden of prompt engineering from the AI system to the builders of MCP servers. This is due to the fact that these prompts specify how best to prompt the AI system so that the LLM utilizes the respective server optimally. For instance, rather than requiring each user to determine the most effective method for prompting a document or transcript summarization, the server can provide tested prompt templates that function reliably across a range of use cases. These are intentionally user-controlled primitives, explicitly triggered through user actions, such as context menu selections or slash commands in a chat window. Each prompt comprises clearly defined messages that provide standardized contexts, supporting predictable and repeatable LLM outputs. Prompts can be dynamically customized via arguments, which allows injecting contextual parameters effectively. Through this, interactions can be tailored to the current user context or operational needs. Servers facilitate the exposure of these prompts through well-defined methods, thereby enabling seamless discovery and invocation. Fig. 9 visualizes the message workflow throughout the MCP architecture in relation to certain prompts exposed by a server. Available protocol operations for prompts include:

- *prompts/list* - used by the client to discover all available prompts of a server. Return value is an array of prompt descriptors.
- *prompts/get* - used to retrieve a specific prompt.

Within the exemplified travel-planning scenario, the AI system might use several available prompt templates to provide them to the user. A certain MCP server for that, might expose a *plan-vacation* prompt, as in Listing 3. By using this template, a user could provide structured input to the AI system such as: *Barcelona, 7 days, $3000, ["beaches", "architecture", "food"]*.

```
{
  "name": "plan-vacation",
  "title": "Plan a vacation",
  "description": "Guide through vacation planning process",
  "arguments": [
    { "name": "destination", "type": "string", "required":
        true },
    { "name": "duration", "type": "number", "description": "
        days" },
    { "name": "budget", "type": "number", "required": false
        },
    { "name": "interests", "type": "array", "items": { "type"
        : "string" } }
  ]
}
```

Listing 3. Prompt template definition for planning a vacation [23].

3) Client: MCP Clients serve as intermediaries within the MCP ecosystem, facilitating stateful, secure and structured interactions between host applications and servers. In essence, they act as a translation layer between an AI application and a MCP server. The distinction to the host is important to clarify. The host is defined as the application with which users interact, while clients refer to the protocol-level components that enable server connections. The clients bear responsibility for invoking tools, querying for resources and propagating prompts that are exposed by a sever. Each client instance is executed within the host, representing a distinct and isolated session with a corresponding server. This design is engineered explicitly to facilitate context exchanges and capability negotiations in a seamless manner. Thus, each client manages a 1:1 relationship with a server. Capability exchange is a core aspect of clients, enabling dynamic, runtime negotiation of available primitives and supported interactions between clients and servers. Through capability declarations, clients articulate their supported functionalities and explicitly inform

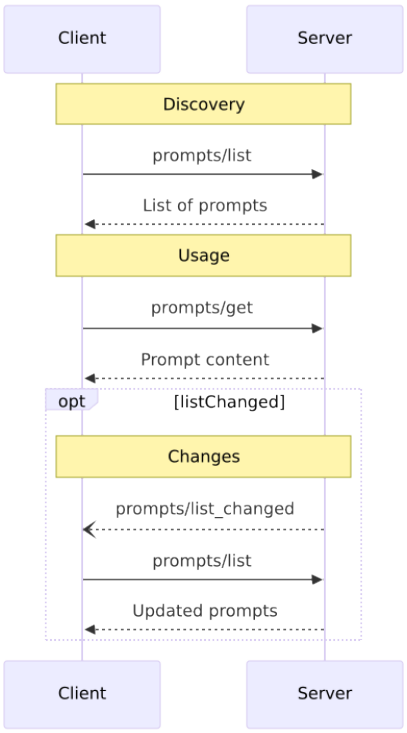


Fig. 9. Message workflow for prompts exposed by a server throughout the MCP architecture [23].

servers about the features they support. This allows servers to tailor their interactions and requests accordingly. Clients are responsible for upholding protocol standards, thereby ensuring that interactions remain robust, secure and efficient across all architectural layers. Stateful management enables clients to dynamically handle context, updates and interaction continuity, thereby significantly enhancing reliability and responsiveness during long-running or multi-step interactions. Clients are responsible for session resilience, including the handling of interruptions, privacy of interactions, timeouts, reconnections and explicit session closures. This management is crucial to prevent resource leaks, enhance security and sustain optimal application performance [23], [50], [52], [53], [55].

Like servers, clients also expose their capabilities through three primitives. These primitives allow builders of MCP servers to create richer interactions.

1) Roots: Clients offer structured representations of file systems or data hierarchies through the concept of roots, thereby defining file system boundaries for server operations. This allows clients to specify which directories servers should prioritize. In this context, roots represent designated areas of the client's data environment that are explicitly exposed to servers for controlled access. Clients manage root lists dynamically, updating server-side contexts in response to changes through the use of notifications. This structured boundary management ensures secure, predictable interactions, preventing unauthorized data access or modification beyond established root directories. Clients must robustly enforce these boundaries, validating all access requests against predefined roots to ensure the optimal safeguarding of data integrity and security. Furthermore, roots consist of file URIs that indicate directories where servers can operate, helping servers understand the scope of available files and folders. Servers typically access files within roots by employing relative paths from the root directories or by utilizing file search tools that respect the root boundaries. Fig. 10 visualizes the message workflow throughout the MCP architecture in relation to certain roots exposed by a client.

Within the exemplified travel-planning scenario, a client might give the connected server access to a specific directory from which it can read a user´s calendar (e.g. *file://Users/agent/travel-planning/calendar*.

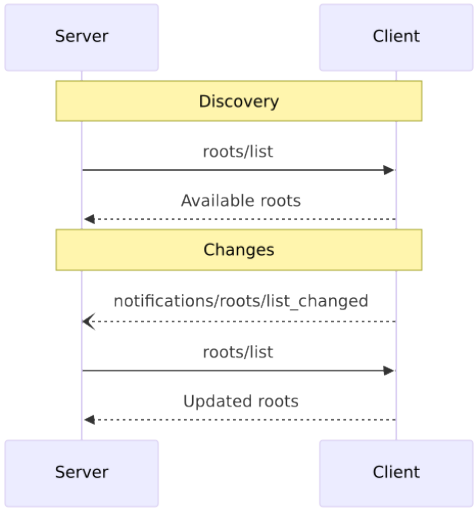


Fig. 10. Message workflow for roots exposed by a client throughout the MCP architecture [23].

2) Elicitation: This primitive allows servers to request additional information from users during interactions, thereby providing a structured method for servers to gather information on demand. Elicitation is particularly useful when builders of servers want to request additional information from the user during a certain step in the workflow of the server (e.g. prior to executing a certain tool) or ask for confirmation of an action. Rather than necessitating the provision of all information at the start of the process or encountering failure when data is absent, servers have the ability to temporarily suspend their operations and request particular inputs from users. This paradigm fosters more adaptable interactions, whereby servers exhibit greater capacity to tailor their responses to user needs, as opposed to adhering to rigid, predefined protocols. Fig. 13 visualizes the message workflow throughout the MCP architecture in relation to a certain elicitation method exposed by a client. This workflow enables dynamic information aggregation. Servers can request specific data when needed, while users provide the respective information through the UI. Consequently, servers resume processing with the newly acquired context.

Within the exemplified travel-planning scenario, a server

might want to ask for the user´s preferences on airplane seats, room type or their contact information to finalize a booking.

3) Sampling: Sampling empowers servers to request AI-generated content, such as text, audio or visual completions from the client's AI application. Clients act as gatekeepers for sampling requests, thereby exercising stringent control over the selection and utilization of LLM resources. Sampling requests are processed via structured JSON-RPC 2.0 methods, clearly specifying model preferences, priorities (e.g. speed, intelligence and cost) and other parameters deemed essential for the precise selection of the model. Clients ensure transparency and user agency by incorporating human-in-the-loop approvals for sampling interactions. This process mitigates risks associated with unauthorized or excessive model usage, thereby maintaining secure, ethical and user-aligned operations. Furthermore, clients are responsible for the parsing of responses, their delivery and the management of errors to ensure the establishment of robust, contextually appropriate interactions. This is especially advantageous when builders of servers seek access to a LLM while maintaining independence of a specific LLM SDK in their server implementation. The utilization of the *sampling/complete* method enables the request of a completion from the client's AI application. This configuration establishes a robust framework in which servers can leverage the intelligence of the connected LLM as an integral component of their processing pipeline, thereby eliminating the necessity of directly managing LLM infrastructure. This approach effectively enables servers to perform AI-dependent tasks without the necessity of direct integration with or financial expenditure on AI models. Instead, servers can request that the client, which already has model access, to handle these tasks on their behalf. Fig. 14 visualizes the message workflow throughout the MCP architecture in relation to sampling methods exposed by a client.

Within the exemplified travel-planning scenario, a server might send a list of available flights it fetched from a API to the LLM to request that it should pick the best-fitting flight for the user from the provided list.

B. Data Layer

The architecture of MCP spans over two distinct layers. This layer defines the JSON-RPC 2.0 based exchange protocol that defines the message structure and semantics for client-server communication. The data layer is the part of MCP which provides the ways for developers to share context from MCP servers to MCP clients and includes [23], [50]–[53]:

1) Lifecycle management: Given its stateful nature, MCP necessitates lifecycle management. The objective of lifecycle management is to facilitate the initialization, negotiation of the capabilities that clients and servers support and termination of the connection between them.

2) Primitives: MCP primitives are a crucial concept within MCP and were explained in detail in the subsection above. They define the capabilities clients and servers can offer each other. These primitives specify the types of contextual information to be shared with AI systems and the range of actions that can be performed. Servers are able to expose the primitives *Tools*, *Resources* and *Prompts*. Each of these offers associated methods for discovery, retrieval and in some cases, execution. Clients utilize these methods to interact with servers, e.g. by using the *tools/list* method to discover all available tools of a specific server. Clients also expose primitives, such as *Sampling*, *Elicitation* and *Logging*. These allow developers of servers to create richer interactions.

3) Other utility capabilities like providing real-time updates and progress tracking for long-running operations. For instance, a server may send tool update notifications to inform connected clients about these changes.

C. Transport Layer

The Transport Layer is responsible for managing the communication mechanisms, authentication and channels that facilitate data exchange between clients and servers. This includes transport-specific connection establishment, message framing and authorization. This layer is responsible for two-way message conversion. In the client-to-server stream, MCP protocol messages are converted into JSON-RPC 2.0 format, allowing for the transport of several data structures and their processing rules. In the reverse server-to-client stream, the received messages in JSON-RPC 2.0 format are converted back into MCP protocol messages. The three JSON-RPC 2.0 message types include *requests*, *responses* and *notifications*. For this layer, MCP supports the transport mechanisms *stdio* and *Streamable HTTP*, both of which transmit messages in JSON RPC 2.0 format. Since MCP is transport-agnostic and can be implemented over any communication channel that supports bidirectional message exchange and preserves the JSON-RPC 2.0 message format and lifecycle requirements, clients and servers may also implement additional custom transport mechanisms to suit specific needs [23], [50], [52], [53].

The stdio transport is particularly well-suited for scenarios where the server process is tightly integrated with the host application. In this mode, the client launches the server as a subprocess and communicates with it through the operating system's standard input and output streams. This facilitates direct process communication between local processes on the same machine. This interaction is strictly confined to valid MCP messages encoded as JSON-RPC 2.0. The server is responsible for reading input from stdin, processing the message and sending responses or notifications via stdout back to the client. This transport method introduces minimal complexity, requiring no network configuration or port management. Additionally, it provides fast, low-latency communication, as data flows directly between local processes running on the same machine without any network overhead. Security is inherently improved through process-level isolation. Subsequently, the stdio transport is used to integrate local resources like local file systems, databases or local APIs through lightweight, synchronous messaging. Through stdio, a local MCP server is typically connected to one MCP client. Fig. 11 illustrates the stdio workflow [23], [50], [52], [53], [55].

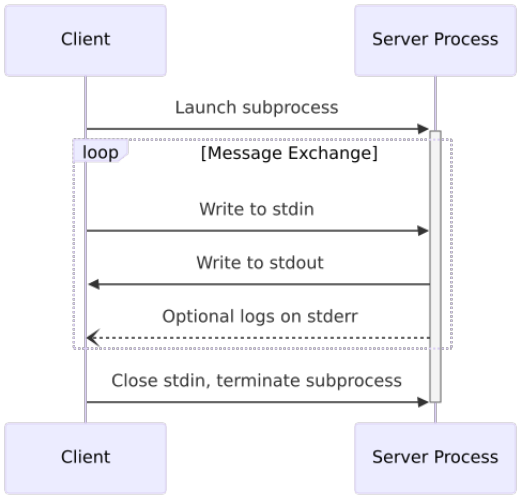


Fig. 11. Message workflow for the stdio transport for a locally running server in MCP [23].

Streamable HTTP represents a contrast to the tight coupling of stdio transport. It uses *HTTP POST* client-to-server messages with optional Server-Sent Events (SSE) [56] for streaming capabilities. In the context of streaming, servers have the capability to emit JSON-RPC 2.0 messages in real time over the SSE stream. This is especially powerful in scenarios involving streaming completions, background computation or progress reporting. With Streamable HTTP, the server operates as a standalone service accessible over HTTP. Clients interact with its endpoint to send JSON-RPC 2.0 messages, initialize sessions, receive real-time updates and process bidirectional communication. It also includes a session management layer which introduces logical isolation and ensures message continuity across long-lived interactions. After initialization, clients may issue requests for prompts, tools or resources. Concurrently, a persistent GET connection is established to facilitate the reception of potential SSE messages from the server, including progress updates, notifications and server-initiated requests. This transport facilitates the integration of remote resources through remote server communication. It supports standard HTTP authentication methods including bearer tokens through OAuth [57], API keys, custom headers and can be introduced to handle multiple asynchronous, event-driven server-calls at once. By this, Streamable HTTP enables remote MCP servers to potentially handle connections to many MCP clients, making it suitable for multi-client deployments. Fig. 12 illustrates the Streamable HTTP workflow [23], [50], [52], [53], [55].

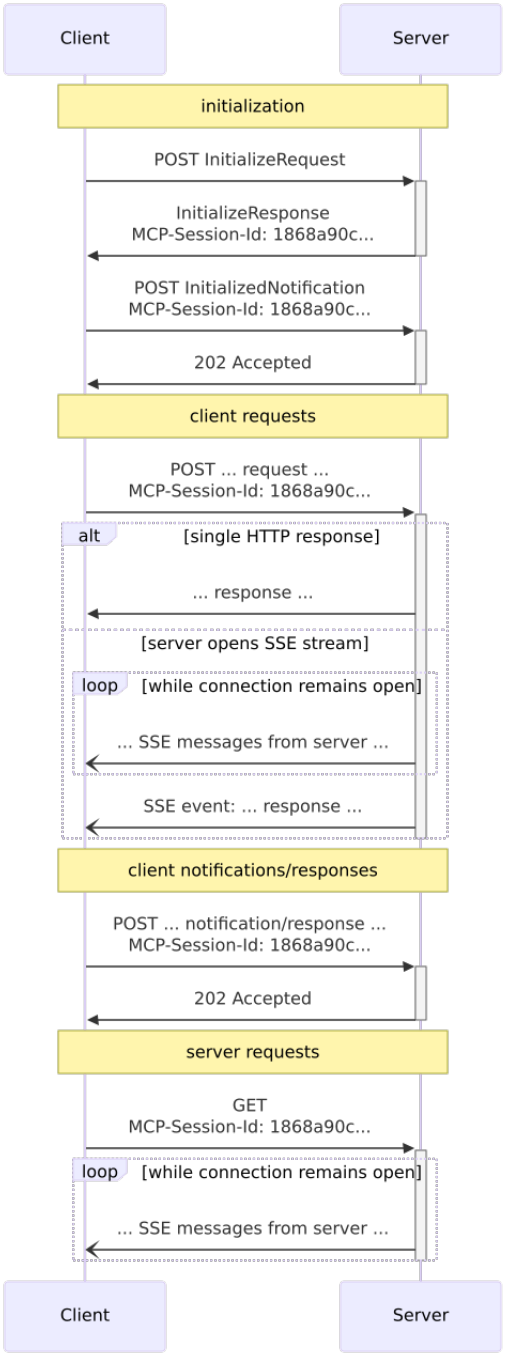


Fig. 12. Message workflow for the Streamable HTTP transport for a remotely running server in MCP [23].

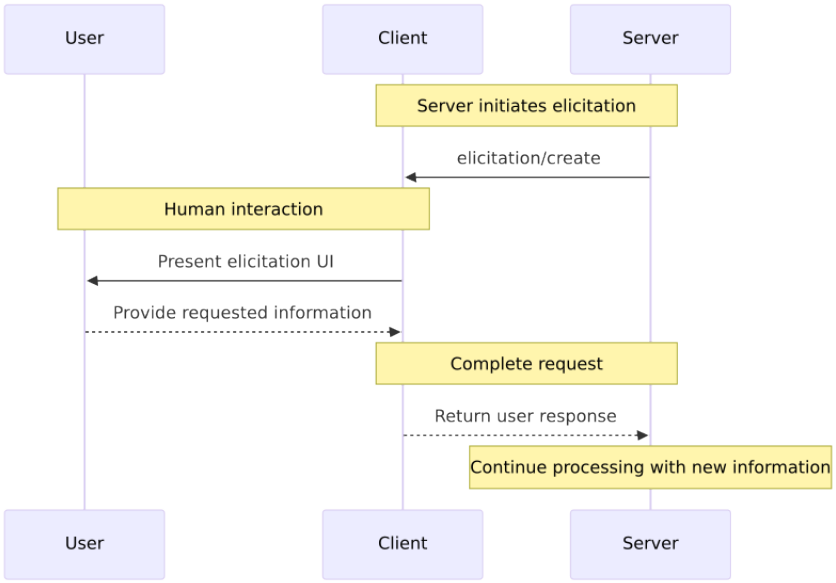


Fig. 13. Message workflow for elicitation methods exposed by a client throughout the MCP architecture [23].

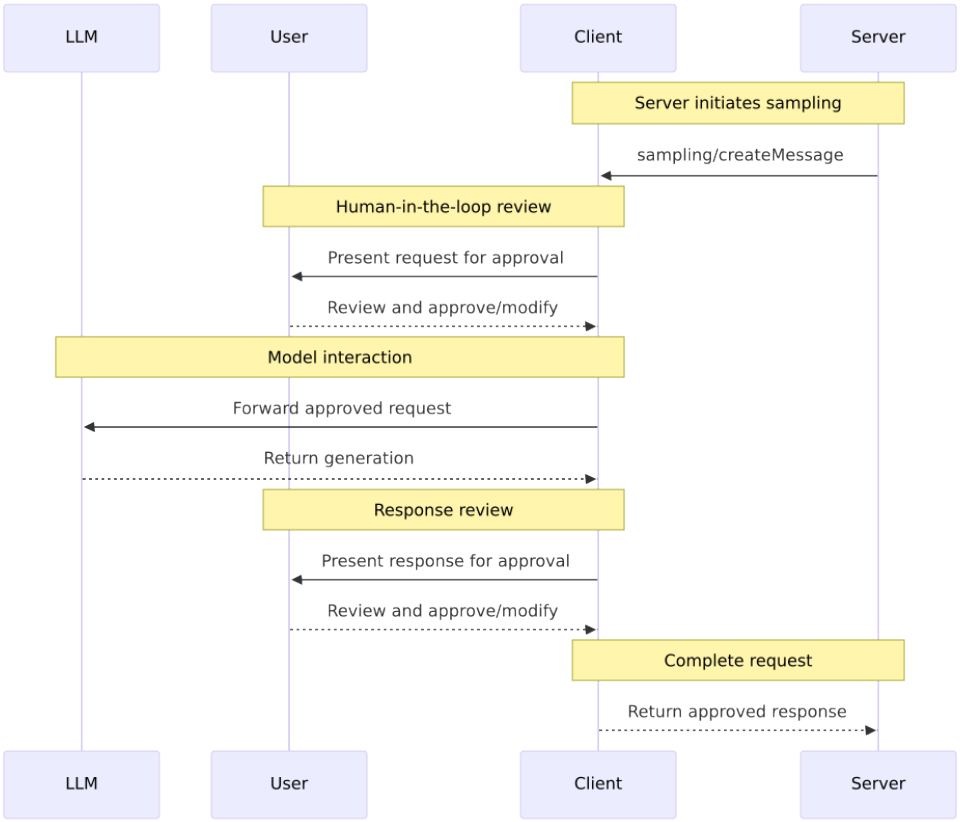


Fig. 14. Message workflow for sampling methods exposed by a client throughout the MCP architecture [23].

IV. Agent-to-Agent Protocol

AI Agents offer a unique opportunity to help people and enterprises be more productive by autonomously handling many daily recurring or complex tasks, enabling them to scale, automate and enhance processes throughout the workplace. As AI Agents become increasingly prevalent, their capacity to interoperate is crucial for the development of sophisticated, multifunctional systems of Agentic AI.

AI Agents can be built by various vendors and through several different frameworks like LangChain [41] or LangGraph [42], each of which brings in specialized workflows, communication mechanisms and individual ecosystems. For certain tasks, a specific framework might be a better fit than another. Without a proper protocol in place, attempts for multi-agent collaboration necessitate improvised heavy glue code to make independent AI Agents communicate with each other. With no common way to discover peers, pass tasks or reference artifacts, every handshake was bespoke and fragmentation quickly became a critical problem in the face of scaling. Thus, the development of multi-agent systems often introduced significant technical bottlenecks, primarily concerning the efficiency of information dissemination between specialized AI Agents. This represents one of the most significant challenges in the establishment and operation of Agentic AI systems, consisting of multiple AI Agents: the question of how to properly make these individual AI Agents work together as one system. It is absolutely critical for these AI Agents, potentially built on different frameworks and vendors, to be able to collaborate in a dynamic, multi-agent ecosystem across isolated data systems and applications, because only then the proposed benefits of Agentic AI can be maximized. Enabling AI Agents to interoperate with each other, even if they were built by different vendors or in a different framework, increases autonomy and multiplies productivity gains, while lowering long-term costs.

This is exactly where the Agent-to-Agent (A2A) protocol comes in. Google designed it to address the challenges they identified in deploying large-scale, multi-agent Agentic AI systems for their customers [24], [58]–[60]. The A2A protocol is an open communication standard designed to enable seamless interaction and collaboration between independent AI Agents within complex Agentic AI systems, resulting in AI Agents retaining their individual capabilities and acting as autonomous entities while collaborating with other AI Agents. It is designed for multi-agent systems of Agentic AI, with the objective of facilitating interoperability between AI Agents. This is achieved by enabling communication between AI Agents in a structured manner, as well as facilitating secure exchange of information and coordination of actions, without the necessity of custom integration glue code. As a universal translator for agent ecosystems, A2A provides a definitive common language and interaction model that bridges the gap between AI Agents built using disparate frameworks, languages or architectures and owned by different organizations. This interoperability is critical in modern enterprise environments, where AI Agents from various providers should collaborate seamlessly as coherent Agentic AI systems to automate complex workflows and drive unprecedented levels of efficiency and innovation. Built entirely on established enterprise standards, the A2A protocol utilizes HTTP-based JSON messages to facilitate structured, secure exchanges. Specifically, it employs JSON-RPC 2.0 payloads for request-response cycles, Server-Sent Events (SSE) [56] for real-time streaming and standard REST API authentication methods, such as OAuth [57]. By standardizing how autonomous AI Agents communicate, A2A eliminates the need for expensive, custom glue code. This allows AI Agents from different frameworks, such as LangChain [41] and Semantic Kernel [61], to collaborate dynamically without necessary manual integration.

A2A is based on three essential pillars designed for enterprise-grade deployment:

1) Privacy and intellectual property: The protocol treats AI Agents as opaque entities. This opacity ensures that autonomous AI Agents can collaborate without revealing their internal memory, proprietary logic or specific tool implementations, thereby preserving data privacy and protecting intellectual property.

2) Seamless integration: By leveraging familiar protocols like HTTP and JSON, A2A ensures high compatibility with existing technology stacks, lowering the adoption barrier for enterprises.

3) Enterprise security: The protocol is engineered to support robust authentication and authorization mechanisms, thereby ensuring that information exchange remains secure across diverse platforms and cloud environments.

It is important to distinguish that while A2A enables communication, it does not inherently increase the intelligence of individual AI Agents. It rather facilitates the exchange of tasks and results. Effective collaboration among AI Agents still requires careful design in task structuring, shared goal alignment and error handling. Therefore, A2A does not eliminate the need for orchestration intelligence, it simply introduces a standardized way of communicating the orchestration efforts. That is why A2A can not be seen as a replacement of frameworks like LangChain [41], since it does not focus on the agent logic.

By providing a universal interoperability layer, A2A offers the required capabilities to represent a critical enabler for the next phase of Agentic AI infrastructure. It facilitates effectively connecting specialized AI Agents into a Agentic AI system capable of completing complex tasks. Furthermore, it allows organizations to move away from building single, overly complex monolithic AI Agents that have to handle whole workflows. Instead, A2A empowers developers to assemble teams of specialized AI Agents that retain their individual autonomy while working across entire enterprise application estates, regardless of their underlying technologies. Critically, businesses benefit from a standardized method for managing their AI Agents across diverse platforms and cloud environments. This modular approach, in combination with

the provided universal interoperability, has the potential to unlock a more versatile, powerful and innovative era of AI Agent collaboration, which is essential for fully realizing the innovational potential of Agentic AI [24], [58]–[60].

A. Architecture

A2A facilitates communication between a Client Agent and a Remote Agent, both being AI Agents. The Client Agent is responsible for formulating and communicating tasks, while the Remote Agent is responsible for acting on those tasks in an attempt to provide the correct information or take the correct action. It is important to acknowledge that any A2A-compliant AI Agent has the capacity to seamlessly transition between roles, thereby facilitating the establishment of flexible topologies, such as peer-to-peer meshes. The A2A protocol architecture, consisting of the Client and Remote Agents, centers around a handful of well-defined, structures that enable Client Agents and Remote Agents to coordinate complex, multi-step workflows in a reliable, extensible manner. These structures encapsulate both the state and the content of AI Agent interactions, while keeping the protocol agnostic to any particular domain or implementation technology and serve as the operational and semantic building blocks for any AI Agent to function within an agent-to-agent ecosystem. These structures, as well as the Client and Remote Agent, are explained in finer detail below [24], [58]–[60], [62].

1) Client Agent: A Client Agent is responsible for delegating requests to Remote Agents on behalf of a user or another system. The user represents the originator of any A2A interaction, embodying the intent or need that sets the workflow in motion. While the user is typically a human end-user, the user can also be a system, service or another AI Agent in hierarchical workflows. Regardless of its form, the user does not directly interact with Remote Agents. Instead, the user relies on the Client Agent to translate the requests and intents into actionable tasks. For this, the Client Agent serves as the initiator of the communication with Remote Agents, employing the A2A protocol to leverage the capabilities of other AI Agents. Subsequent to the submission of a request by a user, the Client Agent constructs a task, encapsulating the user´s intent, relevant metadata and formatted inputs. After identifying a properly fitting Remote Agent to fulfill the task and submitting it to the Remote Agent, the Client Agent manages the exchange of messages and artifacts. For that, bidirectional communication with the Remote Agent is established to send new instructions or follow-ups and to receive outputs along with any intermediate updates. In the context of long-running or stateful interactions, a certain session context is maintained. The Client Agent also takes care of error handling, parsing any failure responses returned by the Remote Agent and executing appropriate recovery strategies such as retries, fallback agent selection or user notifications. When receiving a result, it must be transformed into a user-consumable format and integrated into the surrounding application or user interface. The Client Agent may also handle asynchronous communication through mechanisms such as Server-Sent Events (SSE) [56] or push notifications. For SSE, it establishes a persistent connection and streams updates in real time. If push notification support is enabled, the Client Agent registers with a notification service to receive task updates delivered out-of-band. Altogether, the Client Agent acts as the execution orchestrator, data translator and communication bridge within the A2A protocol, enabling intelligent, context-aware interactions on behalf of the user [24], [59], [60], [63], [64].

2) Remote Agent: The Remote Agent is responsible for exposing an A2A-compliant service endpoint that implements the A2A protocol. It receives requests from Client Agents, executes tasks and responds with results or status updates. A Remote Agent provides one or more skills, which represent discrete operations it can perform. These operations range from simple data retrieval to complex computations or orchestrations involving external APIs or databases. Each skill is formally defined by its input and output schema, thereby enabling consistent invocation across Client Agents. To make its capabilities discoverable, a Remote Agent publishes an agent card. The Remote Agent must also manage its internal resource usage, ensuring fair allocation of compute, memory, network and storage resources during task execution. Alongside execution, it is responsible for enforcing security and access control mechanisms. This includes authenticating Client Agents, verifying message integrity and authorizing access to specific skills based on access policies or token scopes. By abstracting service capabilities into modular, independently managed components, the Remote Agent supports composability, reliability and interoperability within systems of Agentic AI. From the Client Agent´s perspective, the Remote Agent operates as a black-box system, meaning its internal workings, memory or tools are not exposed. Remote Agent is often also referred to as Server, Server Agent or A2A Server [24], [59], [60], [63], [64].

3) Agent Card: Agent cards function as a form of self-description and discovery mechanism. Through these, AI Agents can introduce themselves to the ecosystem. In essence, these are JSON metadata files published by Remote Agents, typically publicly available at */.well-known/agent.json*. These files encompass information such as the Remote Agent´s identity, description, skills, service endpoint (URL), version and authentication requirements. An agent card also declares operational capabilities through a dedicated capabilities section, where boolean values specify whether the agent supports features like real-time streaming updates, asynchronous push notifications or tracking of state transitions. Advertising these capabilities enables Client Agents to adapt their communication strategy accordingly, opting for streaming in cases where low-latency is crucial or relying on push-driven updates when event-based operation is preferable. Agent cards facilitate the discovery process for Client Agents, allowing them to identify a relevant Remote Agent, gain an understanding of its functionality and learn how to interact with it securely and effectively. Client Agents also parse this information to determine whether a Remote Agent is suitable for a given task. Basically, agent cards serve as business cards advertising

an AI Agent´s capabilities that allow AI Agents to discover each other and identify the best AI Agent to perform a certain task. As the primary entry point for coordination, an AI Agent without an agent card is effectively invisible within the A2A-utilizing system [24], [59], [60], [63], [64].

4) Task: The initiation of tasks is executed by Client Agents when a particular Remote Agent is requested to execute work. Consequently, the communication between a Client Agent and a Remote Agent is oriented towards task completion, with the objective of fulfilling end-user requests. In this context, tasks are defined as the fundamental unit of work necessary to fulfill a request. They specify the skill to be executed, along with the input parameters and contextual metadata. Tasks are stateful and progress through a defined lifecycle (submitted, working, input-required, completed, failed). For that, the *state* property moves through a well-defined finite state machine: *submitted* upon receipt, *working* while being processed, *input-required* if user input is needed, *auth-required* if authentication is necessary to proceed and one of the terminal states — *completed*, *failed*, *canceled* or *rejected*. Each transition can carry a nested message to supply human-readable or structured guidance. Remote Agents may also include diagnostic codes or performance metrics in an extended metadata map inside task status, such as estimated remaining time or CPU utilization, empowering Client Agents to make load-management decisions. Tasks also facilitate the tracking of long-running operations and enable multi-turn interactions and collaboration. Consequently, a task can be executed immediately or, for long-running tasks, each of the AI Agents can communicate to maintain synchronization with each other regarding the status of task completion, thereby facilitating asynchronous or synchronous collaboration. According to the established retention policies, tasks are retained on the Remote Agent. As a result, Client Agents are able to reconnect after a significant period of time, such as days, and retrieve or resume conversations as long as the tasks remain within the Remote Agent´s storage window. The output of a task is referred to as an artifact [24], [59], [60], [63], [64].

5) Message: Messages are defined as the medium through which all conversational and control data is transferred outside of final artifacts. They are also understood to depict a single exchange or turn of communication between a Client Agent and a Remote Agent. This communication is characterized by its content and a unique message ID. A message conveys instructions, prompts, task submissions, context, questions, answers or status updates that are not necessarily formal artifacts. It contains one or more parts, which are granular containers for the content itself. Depending on the sender, each message has an associated role, which can either be a "agent" role for messages sent by the Remote Agent or a "user" role for messages sent by the Client Agent. Without messages, inter-agent interaction would not be possible, making them foundational to the A2A protocol [24], [59], [60], [63], [64].

6) Artifact: An artifact is defined as a tangible output generated by a Remote Agent as a result of a Task (e.g. a document, image or structured data), representing the actual deliverables. Artifacts are composed of a unique ID, a human-readable name and an array of parts. This configuration effectively delivers the concrete results of a Remote Agent´s work back to the Client Agent while ensuring structured and retrievable outputs. Artifacts exhibit a close association with the task lifecycle and can be streamed incrementally to the Client Agent. Artifacts represent the materialized knowledge or value created through agentic collaboration [24], [59], [60], [63], [64].

7) Part: Parts are the fundamental content container used within messages and artifacts. A part signifies the most minimal unit of content, with the capacity to comprise text content, a file reference (URL or inline bytes), structured data, audio or video content. Consequently, it enables A2A to be modality agnostic. Through that, parts provide flexibility for AI Agents to exchange various content types within messages and artifacts [24], [59], [60], [63], [64].

B. Protocol Workflow

The high-level workflow of the A2A protocol encompasses an user of an Agentic AI system initiating a task or request, typically without the necessity of comprehending or directly interacting with the underlying AI Agents. The Client Agent, upon receipt of the request, undertakes a thorough analysis to determine the intended objective. This analysis is followed by the identification of a suitable Remote Agent, a process that entails a careful examination of the advertised capabilities, as detailed in the agent cards. After a Remote Agent has been selected, the Client Agent engages the Remote Agent to execute the task. This execution includes authentication, coordination of message exchanges and retrieval of artifacts. The artifacts are then delivered back to the user [24], [59], [62], [64].

This A2A workflow follows three, clearly specified, distinct steps, each of which is listed and explained with more detail below:

1) Discovery: The A2A workflow is initiated when an entity (a human user or another AI Agent) makes a request to the Client Agent. For instance, a user might request assistance in planning a travel trip. To collaborate, AI Agents need to first find each other and understand their capabilities. In a distributed ecosystem of autonomous AI Agents, the ability to locate and engage collaborators dynamically is fundamental. For this, a Client Agent proceeds with the process of discovery. This process entails the identification of available Remote Agents and the retrieval of their agent cards. The purpose of this endeavor is to determine the most suitable Remote Agent for the task at hand. As previously stated, A2A employs a standardization of agent self-descriptions through the agent card, however, various strategies exist for a Client Agent to discover these cards. Each strategy strikes a different balance between accessibility, governance and security, enabling organizations to tailor agent-onboarding to their operational needs. Thus, the selection of a given strategy depends on the deployment environment and security requirements [24], [59], [62], [64]. Fig. 15 visualizes the message workflow throughout the A2A

architecture in relation to discovery. The supported discovery strategies are presented below:

1.1) Open discovery: The simplest and most transparent mechanism is open discovery, which leverages a well-known URL convention. This approach is recommended for public AI Agents or for those intended for broad discovery within a specific domain. Remote Agents make their Agent Card accessible by hosting it at a standardized, well-known URI on their domain. The standard path is *https://agent-server-domain/.well-known/agent-card.json*. Any Client Agent capable of resolving the domain of a potential Remote Agent (e.g. via DNS) is able to perform a HTTP GET request to the respective endpoint. In the event that the agent card is both present and accessible, the Remote Agent will return it as a JSON response. This strategy reduces the discovery process to the steps of the identification of a domain hosting the Remote Agents service and the retrieval of these agent card at the well-known location. This approach aligns with the existing ".well-known" approach adopted by numerous internet protocols, thereby ensuring open discovery benefits from broad compatibility with web crawlers, search engines and off-the-shelf HTTP clients.

1.2) Registry-based discovery: Enterprises that require more curated control over the trustworthiness and suitability of Remote Agents can benefit from a registry-based discovery model. This strategy entails the registration of Remote Agents in an internal catalog, which is managed by an administrator or a governance team. This approach ensures that Remote Agents are not publicly exposed on the internet. The registry may expose a REST API or a dashboard interface through which Remote Agents can be listed, tagged, versioned and assigned metadata to, such as compliance labels or usage quotas. Through this, the curated registry functions as a central repository, maintaining a collection of agent cards. This enables Client Agents to query the registry´s API for a list of approved agent cards to search for Remote Agents based on various criteria (e.g., skills offered, tags, provider name, capabilities). The registry returns matching agent cards or references. This curated process enables organizations to enforce policies around certification, deprecation timelines and domain-specific compliance, while still preserving the automated lookup semantics of A2A.

1.3) Private Discovery: In scenarios where agentic ecosystems must remain entirely private, for instance, due to the use of proprietary algorithms or the presence of sensitive data, a private discovery API model is recommended. Rather than exposing agent cards over the public internet or in a semi-trusted enterprise catalog, the agent platform may offer custom API endpoints that deliver agent cards exclusively to authenticated Client Agents. These private endpoints have the potential to be supported by corporate service meshes, gateways or on-premise key-value stores. Client Agents must present bearer tokens or other credentials to the discovery endpoint, which will then serve tailored agent card content based on the Client Agent´s identity and authorization level. This mechanism facilitates fine-grained segmentation. AI Agents are divided into subsets, with each subset designated for a specific business unit or team within an organization. Given that private discovery frequently relies on bespoke APIs, A2A delegates the implementation details to the organization, focusing instead on the semantics of agent card consumption rather than mandating a specific registry schema.

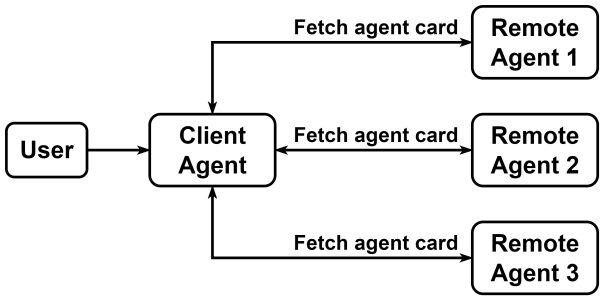


Fig. 15. Discovery workflow in the A2A protocol [59].

2) Authentication: Once the Client Agent identifies a Remote Agent capable of fulfilling the designated task, it undergoes authentication according to the security scheme indicated in the agent card of the Remote Agent. A2A supports security schemes aligned with the OpenAPI specification [65], such as API keys, OAuth [57] and OpenID Connect Discovery [66]. Following the successful authentication of the Client Agent, the Remote Agent becomes responsible for the authorization process and the granting of access control permissions [24], [59], [62], [64]. Fig. 16 visualizes the message workflow throughout the A2A architecture in relation to authentication.

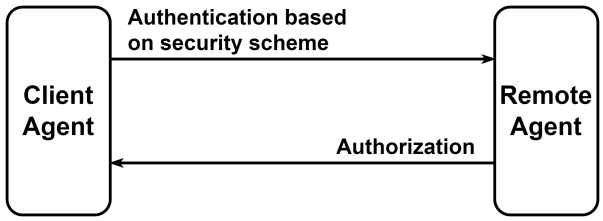


Fig. 16. Authentication workflow in the A2A protocol [59].

3) Communication: The communication process is initiated by the Client Agent, who allocates a task to the designated Remote Agent. Agent-to-agent communication is facilitated over HTTPS for secure transport, with JSON-RPC 2.0 serving as the format for data exchange. Subsequently, the Remote Agent processes the task. In the event that further information is required, the Client Agent is notified and requested to provide additional details. Upon completion of the task, the Remote Agent transmits a message to the Client Agent, accompanied by any generated artifacts. A2A also provides task management features for more complex tasks that can not be completed immediately, such as those requiring human intervention or involving multiple steps. In the case of long-running tasks that extend over hours or days, or in the case of a Client Agent disconnection, the A2A protocol allows

for asynchronous updates through push notifications sent to a secure client-supplied webhook. For large or long outputs or continuous status updates, the A2A protocol supports real-time streaming using Server-Sent Events (SSE) [24], [56], [59], [62], [64]. Fig. 17 visualizes the message workflow throughout the A2A architecture in relation to communication.

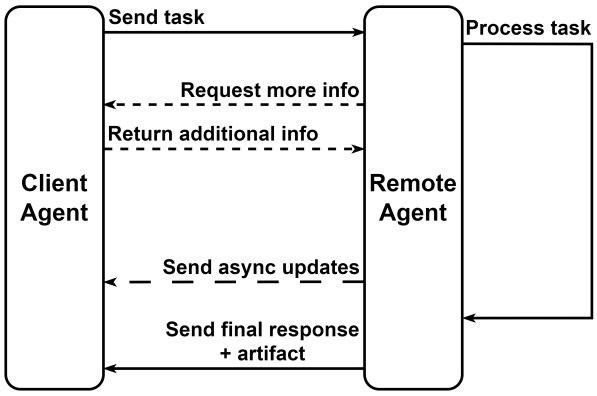


Fig. 17. Communication workflow in the A2A protocol [59].

V. Agent-User Interaction Protocol

In the rapidly evolving landscape of AI-driven applications, the seamless integration between autonomous AI Agents and modern frontend applications is crucial, yet presenting unique challenges. The transition from rudimentary text-generation bots to autonomous AI Agents questions the traditional way of building agentic ecosystems, wherein the transmission of a request is typically answered by receiving a corresponding response. Modern AI Agents, technically their underlying models, reason, plan, use tools and might take minutes to finish a task. They emit progress signals and request user permissions before updating a certain dashboard in real-time, for instance. This demands dynamic and stateful interactions. In addition, popular agentic frameworks like LangChain [41], crewAI [54] and Mastra [67] each build on different communications patterns. The integration of these systems with frontend applications necessitates the implementation of customized integrations, particularly in the absence of a prevailing standard. Attempting to apply the traditional request-response REST API calls to user-facing AI Agents often fails since they exhibit characteristics that make them challenging to serve behind traditional REST APIs. AI Agents introduce long-running processes and streaming of intermediate work. Furthermore, AI Agents might mix structured and unstructured inputs and outputs (e.g. text and audio as well as tool calls). This results in heavy custom code that ties frontend applications tightly to agentic backends, making it hard to switch tools and frameworks and even more difficult to scale the ecosystem. Therefore, there is a necessity for a common way to dynamically stream partial updates to frontend applications so the user is not looking at a static UI while the agentic backend works on long-running tasks and for developers to rely on a shared communication pattern [25], [68]–[70].

The Agent-User Interaction (AG-UI) protocol is an open, lightweight protocol that standardizes the way AI Agents connect to frontend applications and addresses the stated issues from above. It can be regarded as a common language that eliminates the need for custom integration code by enabling bidirectional communication between any agentic backend and any user-facing application. This eliminates the need for framework-specific adapters, letting developers prioritize feature development rather than reinventing communication layers. AG-UI employs an event-based architecture in which AI Agents stream updates while working through standardized event types to represent different states of AI Agents execution, as opposed to imposing the conventional request-response pattern on them.

Fig. 18 visualizes the conceptual difference between traditional REST API calls and the AG-UI event-stream. Instead of waiting for a final response, the AI Agent emits a steady stream of events that describe what it is currently working on.

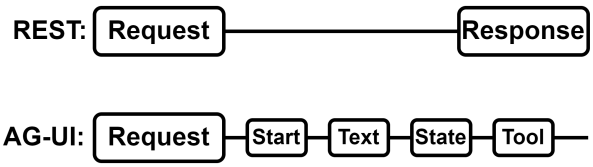


Fig. 18. Comparison of the request-response pattern in REST APIs to the streaming pattern in AG-UI.

AG-UI has been developed with a focus on simplicity and flexibility. This standardization facilitates the flow of information between AI Agents and frontend applications, including the state of the AI Agent, UI intents and user interactions. This implies that the UI can show progress in real-time, whether the AI Agent is generating text, calling tools or updating shared state, enabling real-time, stateful collaboration between AI and human users. Moreover, the protocol is not limited to text. It handles multiple modalities as streaming of images, audio and file attachments alongside the conversation. The standardization of thought processes is also achieved, thereby distinguishing the internal cognitive processes of the AI Agent from the final public response. That enables a pattern in which intermediate steps can be surfaced separately from the final response, allowing frontend applications to decide whether to show users execution progress or simply the end result. AG-UI builds on foundational protocols like HTTP, Server-Sent Events (SSE) [56] or WebSockets, making it easy to adopt regardless of the utilized tech stack. AG-UI transmits a continuous sequence of JSON-formatted events, where each event carries a type field that identifies the action taking place and a streamlined payload containing just the necessary information. The AI Agent emits these events the moment they happen, whether it is generating text tokens, executing functions or modifying application state. The frontend application processes them instantly without repeatedly checking for updates or blocking while waiting for final results, creating a fluid experience where every new

development in state appears as it unfolds. This approach minimizes latency by enabling asynchronous, bidirectional communication between AI Agents and frontend applications. Each event, whether a user input, agent command or UI update, constitutes a compact message that triggers immediate action, thereby ensuring seamless, responsive experiences such as streaming chat responses or live dashboard updates.

Through AG-UI, users watch the AI Agent think, see which tools it is calling and observe results building incrementally. This visibility transforms AI Agents from mysterious background processes into active collaborators, making it possible for humans and AI to work together on evolving documents, plans and outputs. Also, AG-UI implements a full handshake with the A2A protocol and MCP for seamless, full-featured integration [25], [68]–[75].

A. Architecture

AG-UI employs an event-driven, client-server architecture that standardizes communication between AI Agents and frontend applications. For that, a bidirectional event-driven connection between the frontend application and the agentic backend is created, thereby facilitating real-time interaction without the necessity of polling or awaiting for complete responses. This architecture enables AI Agents to stream progress, pause for approvals and synchronize state while users observe the unfolding events in real time.

Fig. 19 visualizes that high-level architecture where AI Agents act as the "backend" that processes user requests and, through AG-UI, generates streaming responses. That bidirectional interaction introduces collaborative workflows where humans and AI Agents work together seamlessly. The backend could also include a secure proxy service that provides additional capabilities, acting as mediator between the frontend application and potentially several AI Agents running in the backend. The AG-UI Client instance is transport agnostic, supporting various transport mechanisms including Server-Sent Events (SSE) [56], webhooks, WebSockets and more. Therefore, AG-UI does not impose how events are delivered. This flexibility lets developers choose the transport that best fits their architecture and makes AG-UI easy to adopt without requiring major changes to existing agentic ecosystems or frontend applications.

A typical interaction cycle comprises the establishment of a connection by the frontend application and the transmission the user´s request. During the processing of the request, the backend emits events as actions happen through this communication channel. These events could represent text chunks, tool calls, state changes or status updates, all delivered immediately as they occur. The frontend application could respond to each incoming event by refreshing the UI, showing partial results or prompting the user for further input, all without having to wait for the entire process to complete. The frontend can send information back to the AI Agents during execution, including user decisions, interface context or cancellation requests. This creates interactive feedback loops where humans and AI Agents collaborate actively. This cycle repeats throughout the AI Agent´s execution, maintaining continuous synchronization between what the AI Agents are doing and what users see through the UI [25], [68], [70], [74].

B. Event types

As mentioned in the section above, the AG-UI protocol utilizes a streaming, event-based architecture. In that, events are the fundamental units of communication between AI Agents and frontend applications, enabling real-time, structured interaction. All of these events share a common set of base properties. The event *type* defines a specific event type identifier. Optionally, a event can hold a *timestamp* indicating when the event was created and a *rawEvent*, containing the original event data. AG-UI organizes agent-user interactions into distinct event categories, each serving a specific purpose in agent-user communication. Below, all supported events are listed within their broader category, accompanied with a detailed description. These events provide a complete vocabulary for everything that happens during interaction, from starting a agent run to streaming text to executing tools. A agent run represents the processing of a user´s request by the respective AI Agent. The goal is to cover almost every possible interaction scenario between AI Agents and frontend applications, so that all of them can be modeled with these events [25], [68], [70], [74].

1) Lifecycle Events: These events represent the lifecycle of an agent run. Lifecycle events provide crucial structure to agent runs, enabling frontends to track progress, manage UI states appropriately and handle errors gracefully. These events form the structural backbone in creating a consistent framework for understanding when operations begin and end, making it possible to implement features like loading indicators, progress tracking and error recovery mechanisms. Fig. 20 visualizes a typical message exchange between an AI Agent and a Client application, which follows a predictable pattern. The exchange begins with a *RunStarted* event and must conclude with either *RunFinished*, indicating success or *RunError*, indicating failure. These events form the boundaries of an agent run. Step events such as *StepStarted* and *StepFinished* are optional and may occur multiple times within a run, allowing for structured, observable progress tracking [25], [68], [70].

1.1) *RunStarted*: The *RunStarted* event is the first event emitted when an AI Agent begins processing a request, thus, signaling that the AI Agent has received the user´s request and initiated working on it. It establishes a new execution context identified by a unique *runId*. This event serves as a marker for frontends to initialize UI elements such as progress indicators or loading states. It also provides crucial identifiers that can be used to associate subsequent events with this specific run.

1.2) *RunFinished*: The *RunFinished* event indicates that an AI Agent has successfully completed all its work for the current agent run. Upon receiving this event, frontends should finalize any UI states that were waiting on the agent's completion. This event marks a clean termination point and indicates that no further processing will occur in this run unless

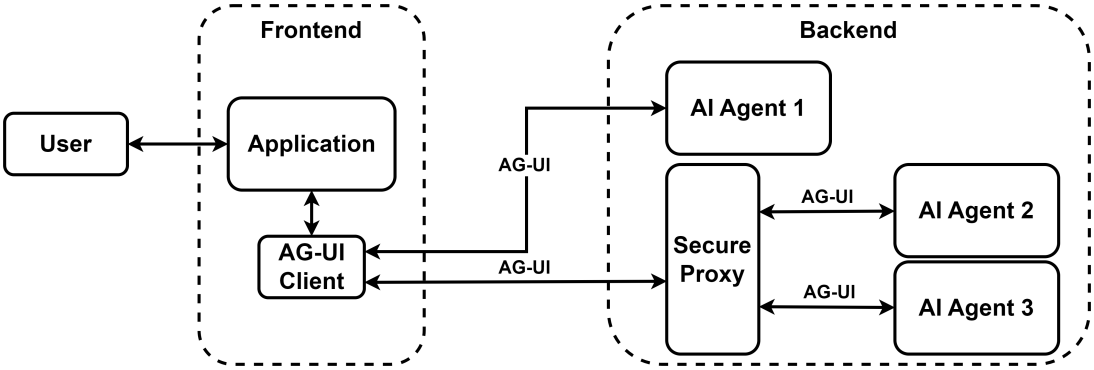


Fig. 19. AG-UI high-level architecture [76].

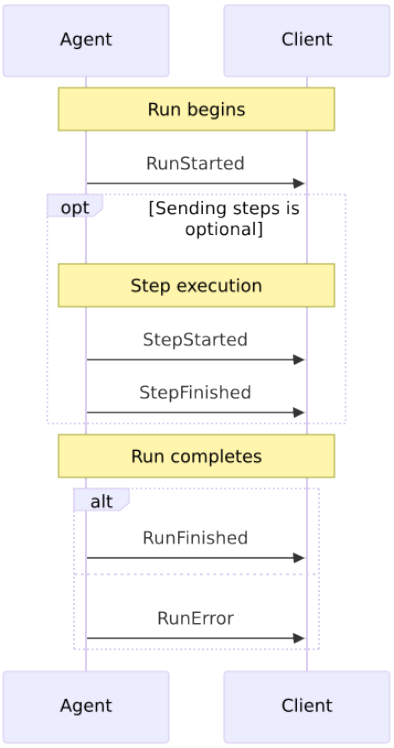


Fig. 20. Conceptual workflow of lifecycle events in the AG-UI protocol [25].

explicitly requested. The optional *result* field can contain any output data produced by the agent run. Essentially, this event indicates that the system is ready for the next request to be processed.

1.3) *RunError*: The *RunError* event indicates that the AI Agent encountered an error it could not recover from, causing the run to terminate prematurely. This event provides information about what went wrong, allowing frontends to display appropriate error messages and potentially offer recovery options. After a *RunError* event, no further processing will occur in this run.

1.4) *StepStarted*: The *StepStarted* event indicates that the agent is beginning a specific subtask or phase of its processing. Steps provide granular visibility into the agent's progress, enabling more precise tracking and feedback in the UI. Steps are optional but highly recommended for complex operations that benefit from being broken down into observable stages. The *stepName* could be the name of a node or function that is currently executing.

1.5) *StepFinished*: The *StepFinished* event indicates that the AI Agent has completed a specific subtask or phase, essentially a step. When paired with a corresponding *StepStarted* event, it creates a bounded context for a discrete unit of work. Frontends can use these events to update progress indicators, show completion animations or reveal results specific to that step. The *stepName* must match the corresponding *StepStarted* event to properly pair the beginning and end of the step.

2) Text Message Events: Text message events represent the lifecycle of text messages in a conversation. These events follow a streaming pattern, where generated text is delivered piece by piece as the AI Agent produces it, therefore handling the streaming of textual content. This creates the familiar "typing" effect popular in chat interfaces, where text appears progressively rather than all at once. Fig. 21 shows that a text message begins with a *TextMessageStart* event, followed by one or more *TextMessageContent* events that deliver chunks of text as they become available and concludes with a *TextMessageEnd* event. The *TextMessageContent* events each contain a *delta* field with a chunk of text. Frontends should concatenate these deltas in the received order to construct the complete message. The *messageId* property links all related events, allowing the frontend to associate content chunks with the correct message. This streaming approach enables real-time display of message content as it is generated, creating a more responsive user experience compared to waiting for the entire message to be complete before showing anything [25], [68], [70].

2.1) *TextMessageStart*: The *TextMessageStart* event signals the start of a new text message in the conversation. It establishes a unique *messageId* that will be referenced by subsequent content chunks and the end event. This event allows frontends to prepare the UI for an incoming message, such as creating a new message bubble with a loading indicator.

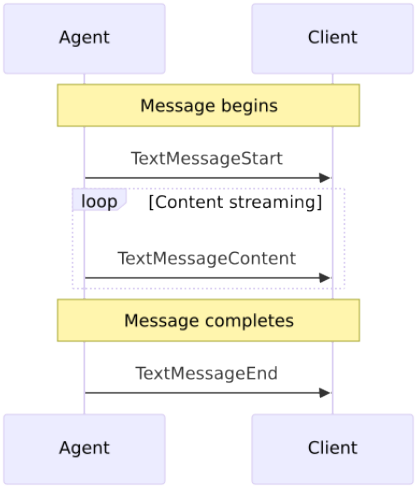


Fig. 21. Conceptual workflow of text message events in the AG-UI protocol [25].

The *role* property identifies whether the message is coming from the assistant or potentially another participant in the conversation.

2.2) *TextMessageContent*: The *TextMessageContent* event delivers incremental parts of the message text as they become available (e.g. word-by-word). Each event contains a small chunk of text in the *delta* property that should be appended to previously received chunks. The streaming nature of these events enables real-time display of content, creating a more responsive and engaging user experience. Implementations should handle these events efficiently to ensure smooth text rendering without visible delays or flickering. It is the responsibility of the frontend application to define how to potentially buffer and display these content pieces.

2.3) *TextMessageEnd*: The *TextMessageEnd* event marks the completion of a streaming text message. After receiving this event, the frontend knows that the message is complete and no further content will be added. This allows the UI to finalize rendering, remove any loading indicators and potentially trigger actions that should occur after message completion, such as enabling reply controls or performing automatic scrolling to ensure the full message is visible.

2.4) *TextMessageChunk*: The *TextMessageChunk* event represents a convenience event that expands from *TextMessageStart* to *TextMessageContent* to *TextMessageEnd* automatically and by that, omits the start and end events. The first chunk for a message must include *messageId* and will emit *TextMessageStart*. If *delta* is defined, a corresponding *TextMessageContent* is emitted. *TextMessageEnd* is automatically emitted when the stream switches to a new message ID or when the stream completes.

3) Tool Call Events: Tool call events represent the lifecycle of tool calls made by AI Agents. In that, they manage these tool executions and provide visibility into tool execution progress in real-time and enable human-in-the-loop workflows where users can approve or reject sensitive operations. Fig. 22 depicts how tool calls follow a streaming pattern similar to text messages. When an AI Agent needs to use a tool, it emits a *ToolCallStart* event, followed by one or more *ToolCallArgs* events that stream the arguments being passed to the tool and concludes with a *ToolCallEnd* event. The *ToolCallArgs* events each contain a *delta* field with a chunk of the arguments. Frontends should concatenate these deltas in the received order to construct the complete arguments object. The *toolCallId* property links all related events, allowing the frontend to associate argument chunks with the correct tool call. This streaming approach allows frontends to show tool executions in real-time, making the AI Agent's actions transparent and providing immediate feedback about what tools are being invoked and with what parameters [25], [68], [70].

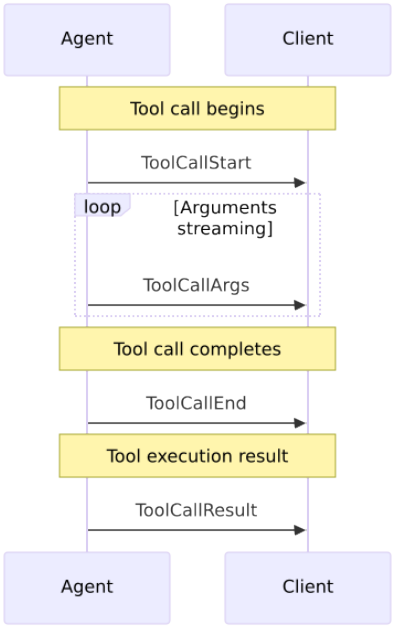


Fig. 22. Conceptual workflow of tool call events in the AG-UI protocol [25].

3.1) *ToolCallStart*: The *ToolCallStart* event indicates that the AI Agent is invoking a tool to perform a specific function, essentially signaling the start of a tool call. This event provides the name of the tool being called and establishes a unique *toolCallId* that will be referenced by subsequent events in this tool call. Frontends can use this event to display tool usage to users, such as showing a notification that a specific operation is in progress. The optional *parentMessageId* allows linking the tool call to a specific message in the conversation, providing context for why the tool is being used.

3.2) *ToolCallArgs*: The *ToolCallArgs* event delivers incremental parts of the tool's specific arguments as they become available. Each event contains a segment of the argument data in the *delta* property. These deltas are often JSON fragments that, when combined, form the complete arguments object for the tool. Streaming the arguments is particularly valuable for complex tool calls where constructing the full arguments may take time. Frontends can progressively reveal these arguments

to users, providing insight into exactly what parameters are being passed to tools.

3.3) *ToolCallEnd*: The *ToolCallEnd* event marks the completion of a tool call. After receiving this event, the frontend concludes that all arguments have been transmitted and the tool execution is underway or completed. This allows the UI to finalize the tool call display and prepare for potential results.

3.4) *ToolCallResult*: The *ToolCallResult* event delivers the output or result from a tool that was previously invoked by the AI Agent. This event is sent after the tool has been executed by the system and contains the actual output generated by the tool. Unlike the streaming pattern of tool call specification (start, args, end), the result is delivered as a complete unit since tool execution typically produces a complete output. Frontends can use this event to display tool results to users, append them to the conversation history or trigger follow-up actions based on the tool's output.

3.5) *ToolCallChunk*: Analogous to 2.4) *TextMessageChunk*, with the difference, that each chunk of *delta* emits a *ToolCallArgs* event in this case.

4) State Management Events: State management events are used to manage and synchronize the AI Agent's state with the state of the frontend application, enabling real-time collaboration on evolving content. State management follows an efficient snapshot-delta pattern where complete state snapshots are sent initially or infrequently, while incremental updates, referred to as deltas, are used for ongoing changes. This approach optimizes for both completeness and efficiency. Snapshots ensure the frontend has the full state context, while deltas minimize data transfer for frequent updates. Together, they enable frontends to maintain an accurate representation of AI Agent state without unnecessary data transmission [25], [68], [70]. Fig. 23 visualizes the workflow of the events playing a part in state management.

4.1) *StateSnapshot*: The *StateSnapshot* event provides a complete snapshot of the AI Agent's current state. This event is typically sent at the beginning of an interaction or when synchronization is needed. It contains all state variables relevant to the frontend, allowing it to completely rebuild its internal representation. Frontends should replace their existing state model with the contents of this snapshot rather than trying to merge it with previous state.

4.2) *StateDelta*: The *StateDelta* event contains incremental updates to the AI Agent's state in the form of JSON Patch operations. Each delta represents specific changes to apply to the current state model. This approach is bandwidth-efficient, sending only what has changed rather than the entire state. Frontends should apply these patches in sequence to maintain an accurate state representation. If a frontend detects inconsistencies after applying patches, it may request a fresh *MessagesSnapshot*.

4.3) *MessagesSnapshot*: The *MessagesSnapshot* event delivers a complete history of messages in the current conversation. Unlike the general state snapshot, this focuses specifically on the conversation transcript. This event is useful for initializing the chat history, synchronizing after connection interruptions

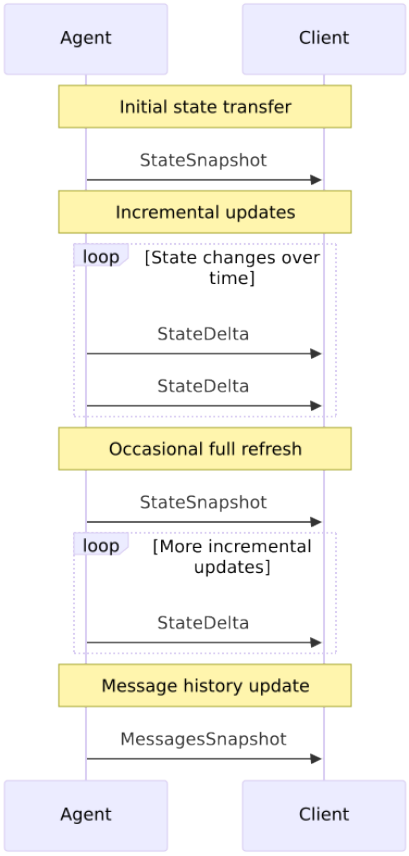


Fig. 23. Conceptual workflow of state management events in the AG-UI protocol [25].

or providing a comprehensive view when a user joins an ongoing conversation. Frontends should use this to establish or refresh the conversational context displayed to users.

5) Activity Events: Activity events expose structured, in-progress activity updates that occur between chat messages, thus, representing ongoing activity progress. They follow the same snapshot/delta pattern as the state management so that UI´s can render a complete activity view immediately and then incrementally update it as new information arrives [25], [68], [70].

5.1) *ActivitySnapshot*: The *ActivitySnapshot* event delivers a complete snapshot of an activity message.

5.2) *ActivityDelta*: The *ActivityDelta* event applies incremental updates to an existing activity using JSON Patch operations.

6) Special Events: Special events provide flexibility by allowing for system-specific custom functionality and integration with external systems. These events do not follow the standard lifecycle or streaming patterns of other event types but instead serve specialized purposes [25], [68], [70].

6.1) *Raw*: The *Raw* event acts as a container for events originating from external systems or sources that do not natively follow the AG-UI protocol. This event type enables interoperability with other event-based systems by wrapping

their events in a standardized format and passing them through. The enclosed event data is preserved in its original form inside the *event* property, while the optional *source* property identifies the system it originates from. Frontends can utilize this information to handle external events appropriately, either by processing them directly or by delegating them to system-specific handlers.

6.2) *Custom*: The *Custom* event provides an extension mechanism for implementing features not covered by the standard event types of AG-UI. Unlike *Raw* events which basically act as pass-through containers, *Custom* events are explicitly part of the protocol but with application-defined semantics. The *name* property identifies the specific custom event type, while the *value* property contains the associated data. This mechanism allows for protocol extensions without the necessity to require formal specification changes. In using them, teams should document their custom events to ensure consistent implementation across frontends and agents.

7) Reasoning Events: Reasoning events support LLM reasoning visibility and continuity, enabling chain-of-thought reasoning while maintaining privacy. These events allow AI Agents to surface reasoning signals (e.g. summaries) and support encrypted reasoning items for state carry, without exposing raw chain-of-thought, if required [25], [68], [70]. Fig. 24 depicts the workflow of the events playing a part in reasoning.

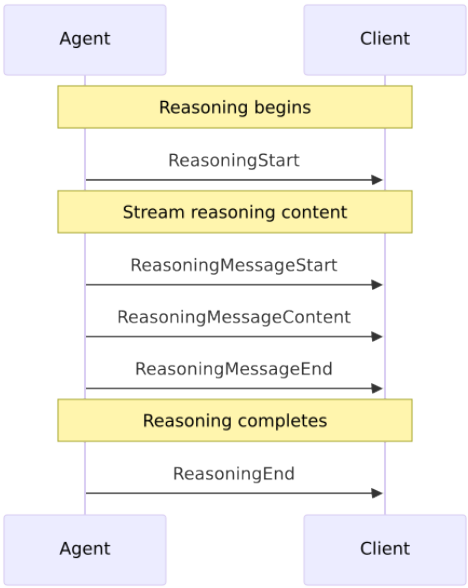


Fig. 24. Conceptual workflow of reasoning events in the AG-UI protocol [25].

7.1) *ReasoningStart*: The *ReasoningStart* event signals that the AI Agent is beginning a reasoning process. It establishes a reasoning context identified by a unique *messageId*.

7.2) *ReasoningMessageStart*: The *ReasoningMessageStart* event signals the start of a streaming reasoning message. This message comprises the visible part of the AI Agent's reasoning that may be displayed to users (e.g. a summary).

7.3) *ReasoningMessageContent*: The *ReasoningMessageContent* event delivers incremental reasoning content to the client. Multiple content events with the same *messageId* should be concatenated to form the complete visible reasoning. Essentially, it represents a chunk of content in a streaming reasoning message.

7.4) *ReasoningMessageEnd*: The *ReasoningMessageEnd* event indicates that all content for the specified reasoning message has been sent, marking the end of a streaming reasoning message. Clients should finalize any UI rendering representing this specific reasoning message.

7.5) *ReasoningMessageChunk*: Analogous to 2.4) *TextMessageChunk*, with the difference, that each chunk with an empty *delta* closes the message.

7.6) *ReasoningEnd*: The *ReasoningEnd* event signals that the AI Agent has completed its reasoning process for the given context, therefore, representing the end of reasoning. No further reasoning events with the same *messageId* should be expected after this event.

7.7) *ReasoningEncryptedValue*: The *ReasoningEncryptedValue* event carries encrypted reasoning content that represents the LLMs internal chain-of-thought related to a specific entity and attaches it to a message or a tool call. This allows the AI Agent to preserve reasoning state across conversation turns without exposing the raw content to the client. The client stores and forwards these encrypted values opaquely, only the AI Agent (or a authorized backend) can decrypt them.

CONCLUSION

This study presented a literature-based evaluation of the evolving landscape of AI Agents and Agentic AI systems, offering a structured taxonomy that highlights foundational concepts, architectural evolution and some exemplified application domains. Beginning with a foundational understanding, AI Agents were characterized as modular, task-specific entities with constrained autonomy and reactivity within the tasks specified for them. Their operational scope is enabled by the integration of LLMs and LVMs, which serve as core reasoning modules for perception, language understanding and decision-making. Additionally, generative AI was positioned as a precursor, establishing the foundation for AI Agents, emphasizing its limitations in autonomy and goal persistence and it was examined how LLMs drive the progression from passive generation to interactive task completion through tool augmentation. This paper then explored the conceptual emergence of Agentic AI systems, representing a paradigm shift as a evolution from isolated AI Agents to orchestrated, multi-agent systems in which multiple specialized AI Agents interact through structured role assignment, task delegation, dynamic allocation of sub-tasks and centralized or distributed control within a broader workflow to achieve a common goal. Stemming from this paradigm shift, this paper examined the MxN integration problem as a key constraint, occurring in several layers in these modern Agentic AI systems.

In order to address this issue, three novel protocols have been introduced in recent years. The agentic ecosystem is

undergoing a rapid organization around this family of open, complementary protocols, with each addressing a distinct layer of interaction: Agent ↔ Tool, Agent ↔ Agent and Agent ↔ UI. These three protocols are complementary and have distinct technical goals, yet a single AI Agent can easily use all three simultaneously to fit into scalable Agentic AI systems. Each protocol has been designed to facilitate specific communication pathways and importantly, to ensure that developers are not constrained to a single vendor, model or interface. These protocols offer the essential flexibility to provide evolution, thereby serving as the unifying element that enables the cohesion of complex workflows.

The Model Context Protocol (MCP) standardizes how AI Agents connect to external systems, such as tools, APIs and resources, to fetch information. The Agent-to-Agent (A2A) protocol provides a standard that defines how AI Agents connect to each other, allowing them to interoperate, collaborate and share work across distributed Agentic AI systems, while the Agent-User Interaction (AG-UI) protocol is designed to provide the general-purpose, bidirectional connection between a user-facing application and any agentic backend. It offers an open, event-based standard, enabling real-time, multi-modal and interactive experiences for end-users.

Fig. 25 visualizes how these protocols connect every part of an Agentic AI system. They dictate how AI Agents access external resources, delegate and collaborate with other AI Agents and how they communicate with end-users through the frontend application. This creates a complete overview where each protocol handles its specialized domain without overlap.

This agentic protocol stack is designed to support various architectural patterns, which can be customized to align with specific organizational requirements and use cases. These patterns leverage different combinations and configurations of the three protocols to create specialized agentic systems. A prevalent pattern is the multi-agent orchestration pattern, wherein a central orchestrator AI Agent delegates specialized tasks to domain-specific AI Agents through A2A. The orchestrator is responsible for maintaining overall workflow control, while specialized AI Agents handle focused activities such as data analysis, content generation or decision support. A typical workflow begins with user input through the frontend application and AG-UI, which triggers a potential orchestrator AI Agent to reason, which could result in coordination with specialized AI Agents through A2A. These AI Agents access tools via MCP and stream results back through AG-UI with real-time updates.

Together, MCP, A2A and AG-UI constitute a comprehensive infrastructure stack that enables a new generation of collaborative systems, in which humans, AI Agents and applications share context, take actions and learn together in real time. This collaborative environment successfully addresses the key challenges in building sophisticated Agentic AI systems. These protocols function as compositional layers that can be deployed independently or in combination depending on specific use case requirements. This modular architecture enables organizations to adopt protocols incrementally. Additionally,

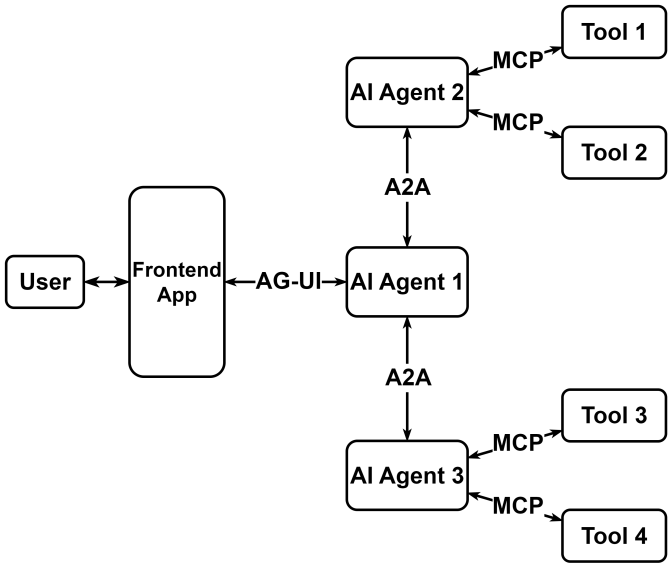


Fig. 25. Sample architecture of an Agentic AI system, showing how the protocols MCP, A2A and AG-UI build the necessary infrastructure for these systems to work upon.

this protocol stack represents a comprehensive architecture for building sophisticated Agentic AI systems that can operate across organizational boundaries while upholding security, transparency and user control. By standardizing tool access, AI Agent collaboration and user interaction, these protocols enable organizations to create modular, scalable and interoperable agentic systems that maintain security and control while maximizing flexibility. Thus, equaling the new infrastructural foundation of Agentic AI.

Statement on AI Usage

Gemini [40] and ChatGPT [11] were utilized to assist on researching relevant topics and articles. DeepL Write [77] was used to examine the grammatical correctness of sections, fitting suggested improvements and detected errors were implemented.

References

[1] A. Newell, J. C. Shaw, and H. A. Simon, "Report on a general problem solving program," in *IFIP congress*, vol. 256, no. 1. Pittsburgh, PA, 1959, p. 1959.

[2] J. McCarthy, "History of lisp," in *History of programming languages*, 1978, pp. 173–185.

[3] H. Moravec, "The stanford cart and the cmu rover," *Proceedings of the IEEE*, vol. 71, no. 7, pp. 872–884, 1983.

[4] R. Calegari, G. Ciatto, V. Mascardi, and A. Omicini, "Logic-based technologies for multi-agent systems: a systematic literature review," *Autonomous Agents and Multi-Agent Systems*, vol. 35, no. 1, pp. 1–67, 2021.

[5] E. H. Shortliffe, *Computer-Based Medical Consultations: MYCIN*, ser. Artificial Intelligence Series. New York: American Elsevier, 1976, vol. 2.

[6] S. Russell and P. Norvig, *Artificial Intelligence, Global Edition A Modern Approach*. Pearson Deutschland, 2021. [Online]. Available: https://elibrary.pearson.de/book/99.150005/9781292401171

[7] K. Hu. (2023) Chatgpt sets record for fastest-growing user base - analyst note. [Online]. Available: https://www.reuters.com/technology/chatgpt-sets-record-fastest-growing-user-base-analyst-note-2023-02-01/

[8] D. B. Acharya, K. Kuppan, and B. Divya, "Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey," *IEEE Access*, vol. 13, pp. 18 912–18 936, 2025.

[9] S. Hosseini and H. Seilani, "The role of agentic ai in shaping a smart future: A systematic review," *Array*, vol. 26, p. 100399, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2590005625000268

[10] G. I. Limited. (2026) Google trends. [Online]. Available: https://trends.google.de/trends/explore?q=Agentic

[11] OpenAI. (2026) Chatgpt. [Online]. Available: https://chatgpt.com/

[12] GitHub. (2026) Github copilot: Your AI pair programmer. GitHub, Inc. [Online]. Available: https://github.com/features/copilot

[13] Jasper AI. (2026) Ai image suite — jasper. Jasper AI, Inc. [Online]. Available: https://www.jasper.ai/image-suite

[14] Midjourney, Inc. (2026) Midjourney. Midjourney, Inc. [Online]. Available: https://www.midjourney.com/

[15] AutoGPT. (2026) Autogpt: Empower your digital tasks. Significant Gravitas. [Online]. Available: https://agpt.co/

[16] Y. Nakajima, "Babyagi: An autonomous task management system," https://github.com/yoheinakajima/babyagi, 2023.

[17] Z. Deng, Y. Guo, C. Han, W. Ma, J. Xiong, S. Wen, and Y. Xiang, "Ai agents under threat: A survey of key security challenges and future pathways," *ACM Comput. Surv.*, vol. 57, no. 7, Feb. 2025. [Online]. Available: https://doi.org/10.1145/3716628

[18] M. Gridach, J. Nanavati, K. Z. E. Abidine, L. Mendes, and C. Mack, "Agentic ai for scientific discovery: A survey of progress, challenges, and future directions," 2025. [Online]. Available: https://arxiv.org/abs/2503.08979

[19] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, "Improving language understanding by generative pre-training," 2018.

[20] Y. Lu, A. Aleta, C. Du, L. Shi, and Y. Moreno, "Llms and generative agent-based models for complex systems research," *Physics of Life Reviews*, vol. 51, pp. 283–293, 2024.

[21] Z. Liu, T. Hoang, J. Zhang, M. Zhu, T. Lan, J. Tan, W. Yao, Z. Liu, Y. Feng, R. RN *et al.*, "Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets," *Advances in Neural Information Processing Systems*, vol. 37, pp. 54 463–54 482, 2024.

[22] R. Sapkota, K. I. Roumeliotis, and M. Karkee, "Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges," *Information Fusion*, vol. 126, p. 103599, Feb. 2026. [Online]. Available: http://dx.doi.org/10.1016/j.inffus.2025.103599

[23] Model Context Protocol Community. (2025) Model context protocol (mcp) – an open standard for ai integration. Model Context Protocol. [Online]. Available: https://modelcontextprotocol.io/

[24] A2A Protocol Community. (2025) A2a protocol — agent2agent (a2a) interoperability standard. A2A Protocol. [Online]. Available: https://a2a-protocol.org/

[25] AG-UI Protocol Contributors. (2026) Ag-ui documentation. https://docs.ag-ui.com/.

[26] A. Radford, I. Sutskever, J. W. Kim, G. Krueger, and S. Agarwal. (2021, Jan.) Clip: Connecting text and images. OpenAI. [Online]. Available: https://openai.com/index/clip/

[27] L. Gabora and J. Bach, "A path to generative artificial selves," in *Progress in Artificial Intelligence*, N. Moniz, Z. Vale, J. Cascalho, C. Silva, and R. Sebastião, Eds. Cham: Springer Nature Switzerland, 2023, pp. 15–29.

[28] A. Bandi, P. V. S. R. Adapa, and Y. E. V. P. K. Kuchi, "The power of generative ai: A review of requirements, models, input–output formats, evaluation metrics, and challenges," *Future Internet*, vol. 15, no. 8, 2023. [Online]. Available: https://www.mdpi.com/1999-5903/15/8/260

[29] OpenAI. (2024, October) Introducing ChatGPT search. OpenAI. [Online]. Available: https://openai.com/index/introducing-chatgpt-search/

[30] P. Bornet and J. Wirtz, *Agentic artificial intelligence: harnessing AI agents to reinvent business, work, and life*. World Scientific, 2025.

[31] M. Chacon-Chamorro, L. F. Giraldo, N. Quijano, V. Vargas-Panesso, C. González, J. S. Pinzón, R. Manrique, M. Ríos, Y. Fonseca, D. Gómez-Barrera *et al.*, "Cooperative resilience in artificial intelligence multiagent systems," *IEEE Transactions on Artificial Intelligence*, 2025.

[32] Y. Talebirad and A. Nadiri, "Multi-agent collaboration: Harnessing the power of intelligent llm agents," 2023. [Online]. Available: https://arxiv.org/abs/2306.03314

[33] A. I. Hauptman, B. G. Schelble, N. J. McNeese, and K. C. Madathil, "Adapt and overcome: Perceptions of adaptive autonomous agents for human-ai teaming," *Computers in Human Behavior*, vol. 138, p. 107451, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0747563222002722

[34] D. Leocádio, L. Guedes, J. Oliveira, J. Reis, and N. Melão, "Customer service with ai-powered human-robot collaboration (hrc): A literature review," *Procedia Computer Science*, vol. 232, pp. 1222–1232, 2024, 5th International Conference on Industry 4.0 and Smart Manufacturing (ISM 2023). [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050924001200

[35] N. Krishnan, "Advancing multi-agent systems through model context protocol: Architecture, implementation, and applications," 2025. [Online]. Available: https://arxiv.org/abs/2504.21030

[36] T. R. McIntosh, T. Susnjak, T. Liu, P. Watters, and M. N. Halgamuge, "The inadequacy of reinforcement learning from human feedback—radicalizing large language models via semantic vulnerabilities," *IEEE Transactions on Cognitive and Developmental Systems*, vol. 16, no. 4, pp. 1561–1574, 2024.

[37] K. I. Roumeliotis, R. Sapkota, M. Karkee, N. D. Tselikas, and D. K. Nasiopoulos, "Plant disease detection through multimodal large language models and convolutional neural networks," 2025. [Online]. Available: https://arxiv.org/abs/2504.20419

[38] OpenAI. (2026) Openai — artificial intelligence research and deployment company. OpenAI. [Online]. Available: https://openai.com/

[39] Anthropic PBC. (2026) Anthropic — ai research and products that put safety at the frontier. Anthropic. [Online]. Available: https://www.anthropic.com/

[40] Google. (2026) Google gemini. [Online]. Available: https://gemini.google.com/app

[41] LangChain, Inc. (2026) Langchain — open-source framework and agent engineering platform for ai applications. LangChain. [Online]. Available: https://www.langchain.com/

[42] ——. (2026) Langgraph — agent orchestration framework for reliable ai agents. LangChain. [Online]. Available: https://www.langchain.com/langgraph

[43] Microsoft Corporation. (2025) Autogen — a framework for building ai agents and applications. Microsoft AutoGen Documentation. [Online]. Available: https://microsoft.github.io/autogen/stable/index.html

[44] L. Peng, D. Li, Z. Zhang, T. Zhang, A. Huang, S. Yang, and Y. Hu, "Human-ai collaboration: Unraveling the effects of user proficiency and ai agent capability in intelligent decision support systems," *International Journal of Industrial Ergonomics*, vol. 103, p. 103629, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0169814124000854

[45] E. Muyale. (2025, September) The m*n integration problem explained and how mcp fixes it. Medium. [Online]. Available: https://medium.com/@edgar_muyale/the-m-n-integration-problem-explained-and-how-mcp-fixes-it-523083a59037

[46] SiteSpeakAI. (2026) What is the N×M integration problem? SiteSpeakAI. [Online]. Available: https://sitespeak.ai/ai-chatbot-terms/nxm-integration-problem

[47] A. Chawla. (2025, June) The m*n integration problem: Solved. Daily Dose of Data Science. [Online]. Available: https://blog.dailydoseofds.com/p/the-mn-integration-problem-solved

[48] U. Barkai and N. Tarbert. (2025, December) The state of agentic UI: Comparing AG-UI, MCP-UI, and A2UI protocols. CopilotKit. [Online]. Available: https://www.copilotkit.ai/blog/the-state-of-agentic-ui-comparing-ag-ui-mcp-ui-and-a2ui-protocols

[49] Anthropic PBC. (2024, Nov.) Introducing the model context protocol. Anthropic. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[50] A. Gutowska. (2025) What is model context protocol (mcp)? IBM Think. [Online]. Available: https://www.ibm.com/think/topics/model-context-protocol#179371084

[51] K. Se. (2025, Mar.) #14: What is mcp, and why is everyone – suddenly! – talking about it? Hugging Face. [Online]. Available: https://huggingface.co/blog/Kseniase/mcp

[52] S. Sharma. (2025, Jun.) Understanding mcp: How the model context protocol solves ai's integration problem. Sajal Sharma's Blog. [Online]. Available: https://sajalsharma.com/posts/understanding-mcp/

[53] P. P. Ray, "A survey on model context protocol: Architecture, state-of-the-art, challenges and future directions," Apr. 2025. [Online]. Available: http://dx.doi.org/10.36227/techrxiv.174495492.22752319/v1

[54] CrewAI, Inc. (2026) Crewai — multi-agent automation platform for ai workflows. CrewAI. [Online]. Available: https://www.crewai.com/

[55] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model context protocol (mcp): Landscape, security threats, and future research directions," 2025. [Online]. Available: https://arxiv.org/abs/2503.23278

[56] MDN Web Docs contributors. (2025, May) Using server-sent events. Mozilla Developer Network (MDN). [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

[57] OAuth.net Community. (2026) Oauth 2.0 — the industry-standard authorization protocol. OAuth.net. [Online]. Available: https://oauth.net/2/

[58] R. Surapaneni, M. Jha, M. Vakoc, and T. Segal. (2025, Apr.) A2a: A new era of agent interoperability. Google Developers Blog. [Online]. Available: https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

[59] R. D. Caballar and C. Stryker. (2025) What is agent2agent (a2a) protocol? IBM Think. [Online]. Available: https://www.ibm.com/think/topics/agent2agent-protocol#684929709

[60] K. Se. (2025, May) #17: What is a2a and why is it – still! – underappreciated? Hugging Face. [Online]. Available: https://huggingface.co/blog/Kseniase/a2a

[61] Microsoft Corporation. (2024, Jun.) Semantic kernel overview. Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/semantic-kernel/overview/

[62] Google Codelabs. (2025) Getting started with agent2agent (a2a) protocol: A purchasing concierge and remote seller agent interactions on cloud run and agent engine. Google Codelabs. [Online]. Available: https://codelabs.developers.google.com/intro-a2a-purchasing-concierge

[63] A. Ehtesham, A. Singh, G. K. Gupta, and S. Kumar, "A survey of agent interoperability protocols: Model context protocol (mcp), agent communication protocol (acp), agent-to-agent protocol (a2a), and agent network protocol (anp)," 2025. [Online]. Available: https://arxiv.org/abs/2505.02279

[64] P. P. Ray, "A review on agent-to-agent protocol: Concept, state-of-the-art, challenges and future directions," May 2025. [Online]. Available: http://dx.doi.org/10.36227/techrxiv.174612014.42157096/v1

[65] OpenAPI Initiative. (2026) Openapi initiative — home of the openapi specification. OpenAPI Initiative. [Online]. Available: https://www.openapis.org/

[66] N. Sakimura, J. Bradley, M. B. Jones, and E. Jay, *OpenID Connect Discovery 1.0*, The OpenID Foundation, Feb. 2014. [Online]. Available: https://openid.net/specs/openid-connect-discovery-1_0.html

[67] Mastra AI. (2025) Mastra — typescript framework for building ai agents and applications. Mastra AI. [Online]. Available: https://mastra.ai/

[68] K. Abdelaty. (2025, Dec.) Ag-ui: A lightweight protocol for agent-user interaction. [Online]. Available: https://www.datacamp.com/tutorial/ag-ui

[69] R. S. (2025, May) Ag-ui protocol: Bridging autonomous agents and interactive interfaces. [Online]. Available: https://medium.com/@ravikumar.singi_16677/ag-ui-protocol-bridging-autonomous-agents-and-interactive-interfaces-bdd21315ea39

[70] Codecademy Team. (2026) Ag-ui: How the agent-user interaction protocol works. [Online]. Available: https://www.codecademy.com/article/ag-ui-agent-user-interaction-protocol

[71] Google A2UI Team. (2025, Dec.) Introducing a2ui: An open project for agent-driven interfaces. https://developers.googleblog.com/introducing-a2ui-an-open-project-for-agent-driven-interfaces/.

[72] Thoughtworks Technology Advisory Board. (2025, Nov.) Ag-ui protocol. https://www.thoughtworks.com/radar/platforms/ag-ui-protocol.

[73] N. Tarbert. (2025, Oct.) Ag-ui is redefining the agent–user interaction layer. [Online]. Available: https://www.copilotkit.ai/blog/ag-ui-is-redefining-the-agent-user-interaction-layer

[74] R. Dannon. (2025, Nov.) Ag-ui: When the frontend got an ai protocol. [Online]. Available: https://medium.com/@roi235/ag-ui-when-the-frontend-got-an-ai-protocol-9223cf904fc4

[75] ag-ui-protocol. (2026) Ag-ui: the agent-user interaction protocol. bring agents into frontend applications. https://github.com/ag-ui-protocol/ag-ui.

[76] AG-UI Protocol Contributors. (2026) Core architecture. https://docs.ag-ui.com/concepts/architecture.

[77] DeepL. (2026) Deepl write. [Online]. Available: https://www.deepl.com/de/write